SISTEMA DE GESTIÓN EMPRESARIAL



UD 00 – Actividades Evaluables 02- Entrevista Técnica

Miriam Mendieta Racedo.-
2DAMS
09/11/2025

ÍNDICE

INTRODUCCIÓN **1**

ENTREVISTA TÉCNICA **2**

PREGUNTA ENTREVISTA 01 2
PREGUNTA ENTREVISTA 02 5
PREGUNTA ENTREVISTA 03 9
PREGUNTA ENTREVISTA 04 11
PREGUNTA ENTREVISTA 05 14
PREGUNTA ENTREVISTA 06 31
PREGUNTA ENTREVISTA 07 53
PREGUNTA ENTREVISTA 08 61
PREGUNTA ENTREVISTA 09 62
PREGUNTA ENTREVISTA 10 68
PREGUNTA ENTREVISTA 11 71
PREGUNTA ENTREVISTA 12 72
PREGUNTA ENTREVISTA 13 74
PREGUNTA ENTREVISTA 14 77
PREGUNTA ENTREVISTA 15 78

BIBLIOGRAFIA **83**

INTRODUCCIÓN

En el mundo laboral actual, especialmente en sectores como la IT, el desarrollo y la gestión de sistemas, las entrevistas técnicas son una herramienta clave para evaluar no solo los conocimientos teóricos del candidato, sino también su capacidad para resolver problemas reales, pensar de forma lógica y comunicarse con claridad.

El presente trabajo forma parte de las actividades evaluables del módulo Sistemas de Gestión Empresarial UD00, y tiene como finalidad preparar al alumnado para una entrevista técnica en el ámbito profesional, centrada en el lenguaje de programación Python 3. A través de la resolución de una serie de ejercicios prácticos, se busca demostrar la capacidad del estudiante para aplicar conocimientos técnicos en situaciones reales:

- Desarrollar habilidades en programación funcional con lambda, map, filter y reduce.
- Validar soluciones de Sudoku mediante lógica y estructuras bidimensionales.
- Analizar patrones en cadenas de texto.
- Simular escenarios como el Buscaminas con conteo de minas adyacentes.
- Diseñar clases orientadas a objetos para modelar entidades como escuelas, profesores y alumnos.
- Implementar persistencia de datos utilizando SQLAlchemy y SQLite.
- Procesar archivos para detectar números palíndromos y primos, incluyendo pruebas unitarias.
- Organizar archivos por extensión en carpetas específicas, con empaquetado en ejecutable mediante PyInstaller.

ENTREVISTA TÉCNICA

PREGUNTA ENTREVISTA 01

Explica y pone un ejemplo sencillo de una función lambda en Python 3.

Después, pon un ejemplo que utilice las funciones de Python:

Utilizando “map( )” (y “list( )” e “int( )” como apoyo para convertir a lista ya números enteros) debe leer de teclado una cadena de texto formada por números separados con espacio y transformarla en una lista formada por números enteros.

Utilizando “filter( )”, elimina de la cadena anterior los números menores que 10.

Con la cadena resultante y utilizando “reduce()”, devuelve la multiplicación de los elementos de la lista.

Una función ***lambda*** es una función anónima, es decir, una función sin nombre, que se define en una sola línea. Se utiliza comúnmente para operaciones simples y rapidas, especialmente como argumento en funciones como map(), filter() o sorted().

Se construyen mediante el operador lambda, los parámetros de la función separados por comas (atención, SIN paréntesis), dos puntos (:) y el código de la función. Esta restringida por la sintaxis a una sola expresión

Ejemplo

```
# Función lambda que suma dos números
suma = lambda x, y: x + y
print(suma(3, 5))  # Salida: 8
```

```python
from functools import reduce  # Importa la función reduce para aplicar operaciones acumulativas sobre listas

# Bucle principal que solicita entrada hasta que sea válida
while True:
    entrada = input("Introduce números separados por espacio: ")

    # Verifica que la entrada no esté vacía ni contenga solo espacios
    if not entrada.strip(): #eliminar los espacios en blanco al principio y final de una cadena.
        print("La entrada está vacía. Inténtalo de nuevo.\n")
        continue
    # Intento de conversión de la entrada en una lista de enteros
    try:
        elementos = entrada.split()  # Divide la cadena por espacios
        numeros = [int(x) for x in elementos]  # Convierte cada elemento a entero
    except ValueError:
        # Si algún elemento no es convertible a entero, se lanza una excepción
        print("Error: Todos los elementos deben ser números enteros separados por espacios.\n")
        continue

    # Filtrado: Selecciona solo los números mayores o iguales a 10
    mayores_que_diez = list(filter(lambda x: x >= 10, numeros))

    # Verifica que haya al menos un número válido para operar
    if not mayores_que_diez:
        print("No hay números mayores o iguales a 10. Inténtalo de nuevo.\n")
        continue

    # Si todas las validaciones se superan, se sale del bucle
    break

# Multiplica todos los elementos filtrados
producto = reduce(lambda x, y: x * y, mayores_que_diez)

# Salida de resultados
print("Lista original:", numeros)
print("Filtrados (>=10):", mayores_que_diez)
print("Multiplicacion de los nº >=10:", producto)
```

```
# Ejemplo de ejecución:
# Introduce números separados por espacio: 5 12 3 20 8
# Lista original: [5, 12, 3, 20, 8]
# Filtrados (>=10): [12, 20]
# Multiplicacion de los nº >=10: 240

# Ejemplo de ejecución:
# Introduce números separados por espacio: 4 7 2
# No hay números mayores o iguales a 10. Inténtalo de nuevo.

# Ejemplo de ejecución:
# Introduce números separados por espacio: 15 abc 25
# Error: Todos los elementos deben ser números enteros separados por espacios.

Introduce números separados por espacio:
La entrada está vacía. Inténtalo de nuevo.

Introduce números separados por espacio: 0 0 0
No hay números mayores o iguales a 10. Inténtalo de nuevo.

Introduce números separados por espacio: 12 dfh 23
Error: Todos los elementos deben ser números enteros separados por espacios.

Introduce números separados por espacio: 1 2 3 4
No hay números mayores o iguales a 10. Inténtalo de nuevo.

Introduce números separados por espacio: 10
Lista original: [10]
Filtrados (>=10): [10]
Multiplicacion de los nº >=10: 10
```

PREGUNTA ENTREVISTA 02

Realiza una aplicación que compruebe si una solución de un Sudoku es correcta o no.

La explicación de las reglas del Sudoku los tiene aquí **https://es.wikipedia.org/wiki/sudoku**

El Sudoku será leído de un archivo "Sudoku.in", formado por 9 líneas de cada una de ellas estará formada por 9 números separados por espacios. El programa nos mostrará por consola y el resultado será mostrado por consola si es correcto o no.

Internamente, el programa realizará la comprobación de si el Sudoku es o no correcto en

en una función definida como "def esSudokuCorrecto(miArrayBi)" que devolverá "True" si

es correcto, "false" de lo contrario.

1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9

Ejemplo salida: incorrecto

Reglas del Sudoku

Un Sudoku clásico consiste en una cuadrícula de 9×9 dividida en 9 bloques de 3×3. Para que una solución sea válida:

- Cada fila debe contener los números del 1 al 9 sin repetir.
- Cada columna debe contener los números del 1 al 9 sin repetir.
- Cada subcuadro 3x3 (hay 9 en total) también debe contener los números del 1 al 9 sin repetir.

```python
#sudoku.py

import os

# ---  Definición de la función de verificación ---

def esSudokuCorrecto(miArrayBi):
    """
    Comprueba si la matriz de Sudoku de 9x9 es válida.
    Un Sudoku es válido si cada fila, cada columna y cada subcuadrícula 3x3
    contiene todos los dígitos del 1 al 9 una sola vez.
    Args:
        miArrayBi (list of list of int): Matriz 9x9 que representa el Sudoku.
    Returns:
        bool: True si el Sudoku es correcto, False en caso contrario.
    """

def esSudokuCorrecto(miArrayBi):
    # Verifica que cada fila tenga números del 1 al 9 sin repetir
    for fila in miArrayBi:
        if set(fila) != set(range(1, 10)):
            return False

    # Verifica que cada columna tenga números del 1 al 9 sin repetir
    for col in range(9):
        columna = {miArrayBi[fila][col] for fila in range(9)}
        if columna != set(range(1, 10)):
            return False

    # Verifica cada subcuadro 3x3
    for fila in range(0, 9, 3):
        for col in range(0, 9, 3):
            subcuadro = set()
            for i in range(3):
                for j in range(3):
                    subcuadro.add(miArrayBi[fila + i][col + j])
            if subcuadro != set(range(1, 10)):
                return False
```

```
    return True


# ---  Lectura del archivo y lógica principal ---

NOMBRE_ARCHIVO = "Sudoku.in"

# Comprobación de que el archivo existe, no vaya a ser que lo estemos buscando bajo las piedras.
if not os.path.exists(NOMBRE_ARCHIVO):
    print(f"Error: No se encontró el archivo '{NOMBRE_ARCHIVO}'.")
    print("Asegúrate de que está en la misma carpeta que el script.")
    exit()

try:
    # Leer el archivo y parsear la matriz
    with open(NOMBRE_ARCHIVO, 'r') as f:
        datos = f.readlines()

    matriz_sudoku = [ ]

    # Procesar cada línea
    for linea in datos:
        # Quitamos espacios iniciales/finales, dividimos por espacios y convertimos a entero
        numeros = [int(n) for n in linea.strip().split() if n.isdigit()]

        # Validación de formato (debe haber 9 números por línea)
        if len(numeros) != 9:
            print("Error de formato: Cada línea en 'Sudoku.in' debe contener exactamente 9 números.")
            # Puedes devolver 'incorrecto' o salir, salimos para ser estrictos.
            exit()

        matriz_sudoku.append(numeros)

    # Validación de formato (debe haber 9 líneas)
    if len(matriz_sudoku) != 9:
        print("Error de formato: 'Sudoku.in' debe contener exactamente 9 líneas.")
        exit()
```

```
    # ---  Llamada a la función y salida ---

    # Hacemos la comprobación
    resultado = esSudokuCorrecto(matriz_sudoku)

    # Imprimimos el resultado por consola
    if resultado:
        print("correcto")
    else:
        print("incorrecto")

except Exception as e:
    # Esto captura cualquier otro error inesperado, como un carácter no numérico en el archivo.
    print(f"Ocurrió un error al procesar el archivo: {e}")
    print("Asegúrate de que 'Sudoku.in' solo contiene números separados por espacios.")

    # desde terminal --> python sudoku.py

\UD00 - Actividad Evaluable 02>  python sudoku.py
Correcto
```

Sudoku.in

5 3 4 6 7 8 9 1 2

6 7 2 1 9 5 3 4 8

1 9 8 3 4 2 5 6 7

8 5 9 7 6 1 4 2 3

4 2 6 8 5 3 7 9 1

7 1 3 9 2 4 8 5 6

9 6 1 5 3 7 2 8 4

2 8 7 4 1 9 6 3 5

3 4 5 2 8 6 1 7 9

PREGUNTA ENTREVISTA 03

Realiza una aplicación que solicite una cadena de texto. El programa debe decir cuantas veces ocurre cada uno de estos patrones sin distinguir mayúsculas y minúsculas :

"00" "101", "ABC", "HO". Un carácter puede formar parte de más de un patrón opuesto.

Por ejemplo: En la cadena "000" el patrón "00" aparece dos veces (una comienza en la posición 0 y otra comienza en la posición 1).

Internamente, el programa realizará la cuenta de patrones con una función definida como "def numeroPatrones(texto)" que devolverá un número entero con el número de patrones propuestos.

```python
import re # módulo de expresiones regulares

def numeroPatrones(texto):
    '''
        Cuenta el número total de ocurrencias de los patrones "00", "101", "ABC" y "HO" en el texto dado,
        incluyendo coincidencias superpuestas y sin distinguir entre mayúsculas y minúsculas.
        Args:
            texto (str): La cadena de texto en la que se buscarán los patrones.
        Returns:
            int: El número total de ocurrencias encontradas de los patrones especificados.
    '''
    # Convertimos el texto a mayúsculas para ignorar diferencias de mayúsculas/minúsculas
    texto = texto.upper()

    # Lista de patrones a buscar
    patrones = ["00", "101", "ABC", "HO"]

    total = 0

    # Para cada patrón, usamos una expresión regular con lookahead para encontrar coincidencias superpuestas
    for patron in patrones:
        # (?=...) es una construcción de lookahead que permite encontrar posiciones donde comienza el patrón sin consumir caracteres
        coincidencias = re.findall(f"(?={patron})", texto)
```

```
        total += len(coincidencias)

    return total

# --- Programa principal ---
cadena = input("Introduce una cadena de texto: ")
resultado = numeroPatrones(cadena)
print("Número total de patrones encontrados:", resultado)


Introduce una cadena de texto: abc00ho101ABCabc00HO101abcABC00ho
Número total de patrones encontrados: 13

Introduce una cadena de texto: Hoy puede ser un gran dia, imposible de recuperar 00
Número total de patrones encontrados: 2

Introduce una cadena de texto:  000101
Número total de patrones encontrados: 3
```

Una expresión regular es una secuencia de caracteres que forma un patrón de búsqueda. Estos patrones se utilizan para buscar y manipular cadenas de texto.

Un lookahead es una técnica avanzada en expresiones regulares que permite verificar si después de una posición específica en el texto aparece un determinado patrón, sin consumir esos caracteres. Es decir, mira hacia adelante para comprobar si hay una coincidencia, pero no la incluye en el resultado.

Tipos de Lookahead

Lookahead Positivo ((?=...)): Comprueba si un patrón específico sigue inmediatamente al punto actual de búsqueda.

Lookahead Negativo ((?!...)): Comprueba si un patrón específico no sigue al punto actual de búsqueda.

PREGUNTA ENTREVISTA 04

Realiza una aplicación que reciba en código mediante un array bidimensional (de longitud variable) un escenario de Buscaminas, donde haya un 0 donde no hay minas y un -1 donde sí existe. Para cada casilla que no tenga una mina, diga cuántas minas adyacentes existen (en diagonal, horizontal y vertical).

Internamente, el programa realizará las acciones con una función definida como "def contandoMinas(miCampo)" que devolverá un array bidimensional con el número de minas adyacentes en cada posición.

Ejemplo Entrada (leído de teclado)
0 0 -1 0

0 -1 -1 0

Ejemplo Salida (mostrar por pantalla)

1 3 -1 2

1 -1 -1 2

Buscaminas (o Minesweeper en inglés) es un juego de lógica en el que el jugador debe descubrir celdas en un tablero sin pisar minas ocultas. El objetivo es revelar todas las celdas que no contienen minas. Si se revela una mina, el juego termina.

El tablero de juego se representa como un **array bidimensional** donde:

- Un 0 o un número positivo representa el número de minas adyacentes.
- Un -1 indica la presencia de una mina.

Ejemplo de Tablero:

[0, 0, 0, 0, -1]

[0, -1, 0, 1, 1]

[-1, 1, -1, 1, 0]

[0, 1, 1, 0, 0]

La adyacencia se refiere a las celdas que se encuentran al lado de una celda específica. En el Buscaminas, cada celda tiene hasta **8 celdas adyacentes** que pueden ser revisadas para contar las minas.

```python
# Función que cuenta las minas adyacentes para cada celda del campo
def contandoMinas(miCampo):
    '''
    Esta función recibe una matriz que representa un campo de Buscaminas,
    donde -1 representa una mina y 0 una casilla vacía.
    Devuelve una nueva matriz donde cada casilla vacía contiene el número
    de minas adyacentes.
    Args:
        miCampo (list of list of int): Matriz del campo de Buscaminas.
    Returns:
        list of list of int: Matriz con el conteo de minas adyacentes.
    '''
    filas = len(miCampo)
    resultado = []

    # Direcciones relativas de las 8 celdas adyacentes
    direcciones = [(-1, -1), (-1, 0), (-1, 1),
                   (0, -1),          (0, 1),
                   (1, -1),  (1, 0),  (1, 1)]

    # Recorremos cada celda del campo
    for i in range(filas):
        fila_resultado = []
        columnas = len(miCampo[i])  # longitud de la fila actual
        for j in range(columnas):
            if miCampo[i][j] == -1:
                fila_resultado.append(-1)
            else:
                contador = 0
                for dx, dy in direcciones:
                    ni, nj = i + dx, j + dy
                    # Verificamos que la fila y columna vecina existen
                    if 0 <= ni < filas and 0 <= nj < len(miCampo[ni]):
                        if miCampo[ni][nj] == -1:
                            contador += 1
                fila_resultado.append(contador)
        resultado.append(fila_resultado)
    return resultado
```

```
# ----------------------
# LECTURA DE DATOS DESDE TECLADO
# ----------------------

print("Introduce el campo de Buscaminas fila por fila.")
print("Usa 0 para casillas vacías y -1 para minas.")
print("Introduce una línea vacía para terminar.")
campo = [ ]  # Lista para almacenar las filas del campo

while True:
    linea = input("Fila: ")
    if linea.strip() == "":
        break  # Si la línea está vacía se termina la entrada
    # línea en una lista de enteros
    fila = list(map(int, linea.strip().split()))
    campo.append(fila)
# Procesamos el campo con la función
resultado = contandoMinas(campo)
# resultado
print("\nResultado:")
for fila in resultado:
    print(' '.join(str(celda) for celda in fila))
```

```
Introduce el campo de Buscaminas fila por fila.
Usa 0 para casillas vacías y -1 para minas.
Introduce una línea vacía para terminar.
Fila: 0 -1 0 -1 0 -1 0 -1 0 -1
Fila: 0 0 0 0 0 0 0 0 -1 0 -1
Fila: -1 -1 -1 -1 -1 -1 -1
Fila: 0 0 0 0 0 0 0 0 0 0
Fila:
Resultado:
1 -1 2 -1 2 -1 2 -1 3 -1
3 4 5 4 5 4 4 3 -1 3 -1
-1 -1 -1 -1 -1 -1 -1
2 3 3 3 3 3 2 1 0 0
```

PREGUNTA ENTREVISTA 05

Diseña la clase "Escuela" para un programa que permita la gestión completa de varias escuelas. Por cada escuela, se instanciará la clase "Escuela" :

- **Clase Escuela**: contendrá la información de las escuelas (nombre, localidad, responsable...) así como de los diferentes profesores y como los diferentes grupos de alumnos, utilizando las siguientes clases propuestas.
    - Debe tener métodos para añadir/eliminar alumnos, profesores

De forma auxiliar a la clase "Escuela", deben crearse las siguientes clases:

- **Clase Profesor:** contendrá información de los profesores que trabajan allí (nombre, tipo(ciencias, letras o mixto))

- **Clase Alumno:** contendrá la información de los alumnos de la escuela (nombre, curso, profesor responsable (sólo un profesor)).

No hay un alumno que vaya a dos escuelas distintas, ni dos profesores que trabajan en diferentes escuelas. Por tanto, elija la estructura de objetos más adecuada para su correcto almacenamiento.
Las clases deben estar declaradas en un archivo separado del programa. Además:

- Se deben representar las relaciones entre clases.
- Deben tener métodos para gestionar correctamente todos los elementos, con sus correspondientes operaciones (inserción, modificación, borrado, visualización, etc.).

Por último, haz un código de ejemplo que permita probar la capacidad de tu diseño

CONSEJO: este tutorial puede ayudarte a organizar las clases Python en varios archivos
https://www.delftstack.com/howto/python/import-a-class-in-python/

Estructura

```
mi_escuela/
│
├── __init__.py      # Marca la carpeta como paquete
├── escuela.py       # Clase Escuela
├── profesor.py      # Clase Profesor
├── alumno.py        # Clase Alumno
└── main.py          # Código para probar
```

```python
# mi_escuela/escuela.py

from typing import Dict
from .profesor import Profesor
from .alumno import Alumno

class Escuela:
    """
    Representa y gestiona una escuela, sus profesores y sus alumnos.
    """

    def __init__(self, nombre: str, localidad: str, responsable: str):
        """
        Inicializa una nueva instancia de Escuela.

        Args:
            nombre (str): Nombre de la escuela.
            localidad (str): Localidad donde se encuentra la escuela.
            responsable (str): Nombre del responsable de la escuela.
        """
        self.nombre = nombre
        self.localidad = localidad
        self.responsable = responsable
        self.profesores: Dict[str, Profesor] = {} # Clave: DNI
        self.alumnos: Dict[str, Alumno] = {}     # CLAVE:  NIA

    # --- Métodos de Gestión de Profesores ---
    def agregar_profesor(self, profesor: Profesor) -> bool:
        """
        Agrega un profesor, usando su DNI como clave única.
        """
        dni = profesor.dni.upper()

        if dni in self.profesores:
            print(f"ERROR: El profesor con DNI {dni} ya existe.")
            return False
```

```
        self.profesores[dni] = profesor
        print(f"INFO: Profesor {profesor.nombre} agregado con éxito (DNI: {dni}).")
        return True

    def eliminar_profesor(self, dni_profesor: str) -> bool:
        """
            Elimina un profesor por su DNI
        """
        dni_profesor = dni_profesor.upper()

        if dni_profesor not in self.profesores:
            print(f"ERROR: Profesor con DNI: {dni_profesor} no encontrado.")
            return False

        del self.profesores[dni_profesor]
        print(f"INFO: Profesor con DNI {dni_profesor} eliminado con éxito.")
        return True

    def modificar_profesor(self, dni: str, nuevo_nombre: str = None, nuevo_tipo: str = None) -> bool:
        """
            Modifica los atributos (nombre y/o tipo) de un profesor por su DNI.
        """
        dni = dni.upper()
        if dni not in self.profesores:
            print(f"ERROR: Profesor con DNI {dni} no encontrado para modificar.")
            return False

        profesor = self.profesores[dni]

        if nuevo_nombre:
            profesor.nombre = nuevo_nombre
        if nuevo_tipo:
            profesor.tipo = nuevo_tipo.upper()

        print(f"INFO: Profesor con DNI {dni} modificado con éxito.")
        return True

    def mostrar_profesores(self):
```

```
        """
        Muestra todos los profesores registrados en la escuela.
        """
        for p in self.profesores.values():
            print(p)


    # --- Métodos de Gestión de Alumnos ---

    def agregar_alumno(self, alumno: Alumno) -> bool:
        """
        Agrega un alumno, usando su NIA como clave única.
        """

        nia = alumno.nia.upper()

        if nia in self.alumnos: # se comprueba si ya existe por NIA
            print(f"ERROR: El alumno con NIA {nia} ya existe.")
            return False


        self.alumnos[nia] = alumno
        print(f"INFO: Alumno {alumno.nombre} agregado con éxito (NIA: {nia}).")
        return True

    def eliminar_alumno(self, nia_alumno: str) -> bool:
        """
        Elimina un alumno por su NIA
        """
        nia_alumno = nia_alumno.upper()

        if nia_alumno not in self.alumnos:
            print(f"ERROR: Alumno con NIA: {nia_alumno} no encontrado.")
            return False

        del self.alumnos[nia_alumno]
        print(f"INFO: Alumno con NIA {nia_alumno} eliminado con éxito.")
        return True
```

```
  def modificar_alumno(self, nia: str, nuevo_nombre: str = None, nuevo_curso: str = None, nuevo_profesor_dni: str = None) -> bool:
      """
         Modifica los atributos (nombre, curso y/o responsable) de un alumno por su NIA.
      """
      nia = nia.upper()
      if nia not in self.alumnos:
         print(f"ERROR: Alumno con NIA/DNI {nia} no encontrado para modificar.")
         return False

      alumno = self.alumnos[nia]

      if nuevo_nombre:
         alumno.nombre = nuevo_nombre
      if nuevo_curso:
         alumno.curso = nuevo_curso

      if nuevo_profesor_dni:
         nuevo_profesor_dni = nuevo_profesor_dni.upper()
      if nuevo_profesor_dni in self.profesores:
         alumno.profesor_responsable = self.profesores[nuevo_profesor_dni]
      else:
         print(f"ERROR: El profesor con DNI {nuevo_profesor_dni} no existe. Responsable no modificado.")
         return False

      print(f"INFO: Alumno con NIA {nia} modificado con éxito.")
      return True

  def mostrar_alumnos(self):

      """
         Muestra todos los alumnos registrados en la escuela.
      """
      for a in self.alumnos.values():
         print(a)
```

```python
# mi_escuela/profesor.py
from typing import Dict
class Profesor:
    """
    Clase que representa un profesor.

    Atributos:
        nombre (str): El nombre del profesor.
        dni (str): Identificador único (DNI) del profesor.
        tipo (str): El tipo de profesor (ciencias, letras, mixto).
    """

    #constructor
    def __init__(self, nombre,dni,tipo): #self es una referencia a la instancia actual de la clase.
        """
        Inicializa un objeto de tipo Profesor.

        Args:
            nombre (str): Nombre del profesor.
            dni (str): Identificador único (DNI) del profesor.
            tipo (str): Tipo de asignatura que imparte.
        """
        self.nombre = nombre
        self.dni = dni.upper() # DNI como clave
        self.tipo = tipo.upper()
    def __str__(self):
        """
            Devuelve una representación en forma de cadena del profesor
        """
        return f"Profesor: {self.nombre}, DNI: {self.dni}, Tipo: {self.tipo}"

    def to_dict(self) -> dict:
        '''
            Devuelve un diccionario con la información del profesor
        '''
        return {"nombre": self.nombre, "DNI": self.dni, "tipo": self.tipo}
```

```python
# mi_escuela/alumno.py

# Usamos un punto para importar desde el módulo hermano 'profesor'
from .profesor import Profesor
from typing import Dict
class Alumno:
    def __init__(self, nombre, nia,curso, profesor_responsable):
        self.nombre = nombre
        self.nia = nia
        self.curso = curso
        self.profesor_responsable = profesor_responsable  # instancia de Profesor

    def __str__(self):
        return f"Alumno: {self.nombre}, NIA: {self.nia}, Curso: {self.curso}, Responsable: {self.profe-
sor_responsable.nombre}"

    def to_dict(self) -> dict:
        return {
            "nombre": self.nombre,
            "nia": self.nia,
            "curso": self.curso,
            "profesor_responsable": self.profesor_responsable.nombre
        }
```

```python
# mi_escuela/main.py

from typing import Dict
import os
import sys

from .escuela import Escuela
from .profesor import Profesor
from .alumno import Alumno
```

```
# Función auxiliar para limpiar la consola
def limpiar_pantalla(menu_name: str):
    """Limpia la consola e imprime el nombre del menú activo."""
    os.system('cls' if os.name == 'nt' else 'clear')
    print(f"--- {menu_name} ---")

# Lista para almacenar todas las escuelas creadas
escuelas = [ ]

# --- Lógica del Menú Principal ---
def menu_principal(escuela_actual: Escuela = None):
    while True:
        limpiar_pantalla("Menú Principal")

        if escuela_actual:
            print(f"ESCUELA ACTUAL: {escuela_actual.nombre} | Profesores: {len(escuela_actual.profeso-
res)} | Alumnos: {len(escuela_actual.alumnos)}\n")
            print("--- Menú de Gestión de Escuela ---")
            print("1. Gestión de Profesores")
            print("2. Gestión de Alumnos")
            print("3. Mostrar Información Completa de la Escuela")
            print("4. Listar Escuelas Creadas")
            print("---")
            print("9. Crear/Cambiar a otra Escuela")
            print("0. Salir del Programa")
        else:
            # Menú si no hay escuela activa
            print("Aún no hay una escuela activa.")
            print("1. Crear una nueva Escuela")
            print("0. Salir del Programa")

        opcion = input("\nSelecciona una opción: ")

        if escuela_actual:
            if opcion == '1':
                menu_profesores(escuela_actual)
            elif opcion == '2':
                menu_alumnos(escuela_actual)
```

```
        elif opcion == '3':
            mostrar_informacion_escuela(escuela_actual)
        elif opcion == '4':
            nueva_escuela = listar_escuelas()  # Llama a la función para listar escuelas
            if nueva_escuela:
                escuela_actual = nueva_escuela  # Cambia a la nueva escuela seleccionada
        elif opcion == '9':
            escuela_actual = crear_o_cambiar_escuela()
        elif opcion == '0':
            print("\nSaliendo del programa. ¡Hasta pronto!")
            sys.exit()
        else:
            input("Opción no válida. Presiona Enter para continuar...")
    else:
        if opcion == '1':
            escuela_actual = crear_o_cambiar_escuela()
        elif opcion == '0':
            print("\nSaliendo del programa. ¡Hasta pronto!")
            sys.exit()
        else:
            input("Opción no válida. Presiona Enter para continuar...")

# --- Lógica de Creación de Escuela ---
def crear_o_cambiar_escuela():
    """
    Solicita los datos para crear una nueva instancia de Escuela.
    """
    print("\n--- Creación de Nueva Escuela ---")
    nombre = input("Nombre de la Escuela: ")
    localidad = input("Localidad: ")
    responsable = input("Nombre del Director/Responsable: ")

    nueva_escuela = Escuela(nombre, localidad, responsable)
    escuelas.append(nueva_escuela)  # Agrega la nueva escuela a la lista
    print(f"\n¡Escuela '{nombre}' creada con éxito!")
    input("Presiona Enter para ir al menú principal...")
    return nueva_escuela
```

```
# --- Función para Listar Escuelas ---
def listar_escuelas():
    """
    Muestra un listado de todas las escuelas creadas y permite seleccionar una escuela.
    """
    print("\n--- Listado de Escuelas ---")
    if not escuelas:
        print("No hay escuelas registradas.")
        return

    for i, escuela in enumerate(escuelas):
        print(f"{i + 1}. {escuela.nombre} | Localidad: {escuela.localidad} | Responsable: {escuela.respon-
sable}")

    # Permitir al usuario seleccionar una escuela
    seleccion = input("\nSelecciona el número de la escuela a la que deseas cambiar (o presiona Enter
para volver): ")

    if seleccion.isdigit() and 1 <= int(seleccion) <= len(escuelas):
        escuela_seleccionada = escuelas[int(seleccion) - 1]
        print(f"\nCambiando a la escuela: {escuela_seleccionada.nombre}")
        return escuela_seleccionada  # Devuelve la escuela seleccionada
    else:
        print("Selección no válida. Regresando al menú principal...")
        return None  # Regresa None si no se elige correctamente

# --- Lógicas de Gestión de Profesores ---

def menu_profesores(escuela: Escuela):
    """
    Muestra el menú de gestión de profesores.
    """
    while True:
        limpiar_pantalla("Gestión de Profesores")
        print(f"GESTIÓN DE PROFESORES en {escuela.nombre}")
        print("1. Añadir Profesor")
        print("2. Eliminar Profesor")
        print("3. Listar todos los Profesores")
```

```
        print("4. Modificar Profesor")
        print("0. Volver al Menú Principal")

        opcion = input("\nSelecciona una opción: ")

        if opcion == '1':
            agregar_profesor(escuela)
        elif opcion == '2':
            eliminar_profesor(escuela)
        elif opcion == '3':
            listar_profesores(escuela)
        elif opcion == '4':
            modificar_profesor(escuela)
        elif opcion == '0':
            return
        else:
            input("Opción no válida. Presiona Enter para continuar...")

def agregar_profesor(escuela: Escuela):
    """
    Solicita los datos y añade un objeto Profesor.
    """
    print("\n--- Añadir Nuevo Profesor ---")
    nombre = input("Nombre del profesor: ")
    #  DNI como clave única
    dni = input("DNI del profesor (Clave Única): ").upper()
    while True:
        tipo = input("Tipo (Ciencias, Letras, Mixto): ").capitalize()
        if tipo in ['Ciencias', 'Letras', 'Mixto']:
            break
        else:
            print("Tipo no válido. Debe ser 'Ciencias', 'Letras' o 'Mixto'.")

    nuevo_profesor = Profesor(nombre, dni, tipo)
    escuela.agregar_profesor(nuevo_profesor)
    input("Presiona Enter para continuar...")
```

```python
def eliminar_profesor(escuela: Escuela):
    """
    Solicita el DNI y elimina un objeto Profesor.
    """
    print("\n--- Eliminar Profesor ---")
    dni = input("DNI del profesor a eliminar: ").upper()
    escuela.eliminar_profesor(dni)
    input("Presiona Enter para continuar...")

def modificar_profesor(escuela: Escuela):
    """
    Solicita el DNI del profesor y los datos a modificar.
    """
    print("\n--- Modificar Profesor ---")
    dni = input("DNI del profesor a modificar: ").upper()

    if dni not in escuela.profesores:
        print("ERROR: Profesor no encontrado. Volviendo al menú.")
        input("Presiona Enter para continuar...")
        return

    profesor_actual = escuela.profesores[dni]
    print(f"\nModificando a {profesor_actual.nombre} (Deje en blanco para no modificar)")
    nuevo_nombre = input("Nuevo Nombre: ")
    nuevo_tipo = input(f"Nuevo Tipo (Actual: {profesor_actual.tipo}): ")

    # Llama al método de Escuela
    escuela.modificar_profesor(dni, nuevo_nombre or None, nuevo_tipo or None)

    input("Presiona Enter para continuar...")

def listar_profesores(escuela: Escuela):
    """
    Muestra la lista de todos los profesores.
    """
    print(f"\n--- Listado de Profesores ({len(escuela.profesores)}) en {escuela.nombre} ---")
    if not escuela.profesores:
        print("No hay profesores registrados.")
```

```
    else:
        #  .values() para obtener los objetos Profesor
        for p in escuela.profesores.values():
            print(f"- {p}")
    input("\nPresiona Enter para continuar...")

# --- Lógicas de Gestión de Alumnos ---

def menu_alumnos(escuela: Escuela):
    """
    Muestra el menú de gestión de alumnos.
    """
    while True:
        limpiar_pantalla("Gestión de Alumnos")
        print(f"GESTIÓN DE ALUMNOS en {escuela.nombre}")
        print("1. Añadir Alumno")
        print("2. Eliminar Alumno")
        print("3. Listar todos los Alumnos")
        print("4. Modificar Alumno")
        print("0. Volver al Menú Principal")

        opcion = input("\nSelecciona una opción: ")

        if opcion == '1':
            agregar_alumno(escuela)
        elif opcion == '2':
            eliminar_alumno(escuela)
        elif opcion == '3':
            listar_alumnos(escuela)
        elif opcion == '4':
            modificar_alumno(escuela)
        elif opcion == '0':
            return
        else:
            input("Opción no válida. Presiona Enter para continuar...")
```

```
def agregar_alumno(escuela: Escuela):
    """
    Solicita los datos del alumno (incluyendo NIA y profesor responsable)
    y añade un objeto Alumno a la escuela.
    """
    print("\n--- Añadir Nuevo Alumno ---")

    if not escuela.profesores:
        print("ERROR: Debes añadir profesores antes de poder asignar un responsable a un alumno.")
        input("Presiona Enter para continuar...")
        return

    nombre_alumno = input("Nombre del alumno: ")
    nia_alumno = input("NIA del alumno (Clave Única): ").upper()
    curso = input("Curso del alumno: ")

    # Asignación de profesor responsable
    print("\nProfesores disponibles (Referencia por DNI):")
    for dni, prof in escuela.profesores.items():
        print(f"  [{dni}] {prof.nombre} ({prof.tipo})")

    dni_responsable = input("DNI del profesor responsable: ").upper()

    if dni_responsable in escuela.profesores:
        profesor_obj = escuela.profesores[dni_responsable]

        nuevo_alumno = Alumno(nombre_alumno, nia_alumno, curso, profesor_obj)

        escuela.agregar_alumno(nuevo_alumno)
    else:
        print(f"ERROR: El profesor con DNI '{dni_responsable}' no existe. Alumno no añadido.")

    input("Presiona Enter para continuar...")

def eliminar_alumno(escuela: Escuela):
    """
    Solicita el NIA y elimina un objeto Alumno.
    """
```

```
    print("\n--- Eliminar Alumno ---")
    nia = input("NIA del alumno a eliminar: ").upper()
    escuela.eliminar_alumno(nia)
    input("Presiona Enter para continuar...")

def modificar_alumno(escuela: Escuela):
    print("\n--- Modificar Alumno ---")
    nia = input("NIA del alumno a modificar: ").upper()

    if nia not in escuela.alumnos:
        print(f"ERROR: Alumno con NIA/DNI {nia} no encontrado.")
        return

    alumno_actual = escuela.alumnos[nia]
    print(f"Modificando a {alumno_actual.nombre} (Deje en blanco para no modificar)")

    nuevo_nombre = input(f"Nuevo Nombre (Actual: {alumno_actual.nombre}): ")
    nuevo_curso = input(f"Nuevo Curso (Actual: {alumno_actual.curso}): ")
    nuevo_profesor_dni = input("Nuevo DNI/NIE del profesor responsable (Deje en blanco para no modificar): ")

    # Llamada a la función de la clase Escuela
    escuela.modificar_alumno(
        nia=nia,
        nuevo_nombre=nuevo_nombre or None,
        nuevo_curso=nuevo_curso or None,
        nuevo_profesor_dni=nuevo_profesor_dni or None
    )


def listar_alumnos(escuela: Escuela):
    print(f"\n--- Listado de Alumnos ({len(escuela.alumnos)}) en {escuela.nombre} ---")
    if escuela.alumnos:
        for alum in escuela.alumnos.values():
            print(f"- Nombre: {alum.nombre}, NIA: {alum.nia}, Curso: {alum.curso}, Responsable: {alum.profesor_responsable.nombre}")
    else:
        print("No hay alumnos registrados.")
```

```python
# --- Función para mostrar información completa ---

def mostrar_informacion_escuela(escuela: Escuela):
    """
    Muestra un resumen completo de la escuela.
    """
    limpiar_pantalla("Información Completa de la Escuela")
    print("========================================")
    print(f"DATOS DE LA ESCUELA: {escuela.nombre}")
    print("========================================")
    print(f"Localidad: {escuela.localidad}")
    print(f"Director/Responsable: {escuela.responsable}")
    print("\nPROFESORES:")
    listar_profesores(escuela)
    print("\nALUMNOS:")
    listar_alumnos(escuela)

    input("\nPresiona Enter para volver al menú principal...")


# --- Inicialización del Programa ---

if __name__ == "__main__":
    menu_principal()

    #en consola python -m mi_escuela.main
```

```
--- Menú Principal ---
Aún no hay una escuela activa.
1. Crear una nueva Escuela
0. Salir del Programa

Selecciona una opción: 1

--- Creación de Nueva Escuela ---
Nombre de la Escuela: Sol
Localidad: Valencia
```

```
Nombre del Director/Responsable: Miriam

¡Escuela 'Sol' creada con éxito!
Presiona Enter para ir al menú principal...

--- Menú Principal ---
ESCUELA ACTUAL: Sol | Profesores: 0 | Alumnos: 0

--- Menú de Gestión de Escuela ---
1. Gestión de Profesores
2. Gestión de Alumnos
3. Mostrar Información Completa de la Escuela
4. Listar Escuelas Creadas
---
9. Crear/Cambiar a otra Escuela
0. Salir del Programa

Selecciona una opción:
```

PREGUNTA ENTREVISTA 06

SQLAlchemy es una biblioteca de Python con varias funciones y drivers para tratar con bases de datos. Una de las funciones más destacadas es la inclusión de un ORM (Object Relational Mapping).

Partiendo del diseño de clases de la pregunta anterior y su código, realiza un nuevo programa que añada persistencia a las clases, manteniendo sus relaciones utilizando el ORM de Python 3 incluido en la biblioteca SQLAlchemy con el driver de SQLite.
Haz un código de ejemplo que permita probar la persistencia adquirida al implementar el ORM con objetos de la clase "Alumno", indicando claramente en comentarios que debe hacerse para para probarla.

Consejo 01: Puede ayudarte en esta tarea estos tutoriales:

a) Para poner en marcha SQLAlchemy ORM:

https://j2logo.com/python/sqlalchemy-tutorial-de-python-sqlalchemy-guia-de-inicio/

b) Para entender relaciones 1 a muchos **https://programmerclick.com/article/38091329500/**
https://www.hectormansilla.cl/2020/05/14/relaciones-uno-a-muchos-en-sqlalchemy/

Consejo 02: una buena estrategia para probar la persistencia es hacer un código/funciones que crean/modifican/eliminan algunos objetos, cerrar el programa, comentar esos fragmentos y comprobar que pese a no ejecutarse los cambios, éstos están reflejados en la persistencia y en el programa.

Consejo 03: para instalar SQLAlchemy, puedes ayudarte de "pip3 install SQLAlchemy".

pip es el gestor de paquetes estándar de Python: terminal →pip -- version

SQLAlchemy es una biblioteca de Python que permite interactuar con bases de datos relacionales de forma flexible y potente, usando tanto SQL directo como programación orientada a objetos.

Un ORM (Mapeo Objeto-Relacional) permite convertir datos entre sistemas incompatibles: objetos en Python ↔ tablas en SQL. Con SQLAlchemy se puede:

- Definir clases como modelos de las tablas.
- Crear relaciones entre ellas (uno a muchos, muchos a muchos, etc.).
- Insertar, consultar, actualizar y eliminar datos usando métodos de Python.
- En el ORM de SQLAlchemy, hay tres componentes esenciales para empezar:
-

Componente	Función
Engine (Motor)	La conexión de bajo nivel con la base de datos (dónde está y cómo conectar).
Base (Base Declarativa)	La clase base que usaremos para definir nuestros modelos (tablas).
Session (Sesión)	El "área de trabajo" donde se guardan, consultan y modifican los objetos. Es la interfaz principal del ORM.

Guía de Puesta en Marcha de SQLAlchemy ORM

1. Configurar el Entorno Virtual
   Navegar a la carpeta raíz del proyecto

2. Crear y activar un entorno virtual

```
python -m venv venv

source venv/bin/activate   # En Linux/macOS

.\venv\Scripts\activate    # En Windows
```

3. Instalar SQLAlchemy

```
pip install sqlalchemy
```

Proyecto_SQLAlchemy es la carpeta raíz donde creaste el entorno virtual:

Proyecto_SQLAlchemy/ <-- CARPETA RAÍZ

├── .venv/ <-- Entorno Virtual (contiene Python y SQLAlchemy)

├── main.py

└── mi_escuela/ <-- PAQUETE de Python

├── __init__.py (Crucial para paquetes)

├── database.py <-- ¡NUEVO! Conexión a SQLite y Base ORM

├── profesor.py (Con código ORM pegado)

└── alumno.py (Con código ORM pegado)

4. Crear database.py

   Este archivo contiene la configuración fundamental para SQLAlchemy.

   Crear database.py dentro de la carpeta mi_escuela.

```python
# mi_escuela/database.py

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

# 1. Definir la URL de la Base de Datos
# Usamos SQLite y especificamos el nombre del archivo.
DATABASE_URL = "sqlite:///escuela.db"

# 2. Crear el Motor (Engine) de la BD
# El 'Engine' es el punto de inicio para la base de datos.
engine = create_engine(DATABASE_URL, echo=True) # 'echo=True' muestra las sentencias SQL generadas

# 3. Crear la Base Declarativa (ORM Base)
# Esta clase Base será heredada por todas nuestras clases de modelo.
Base = declarative_base()
```

```python
# 4. Crear el Constructor de Sesiones
# La 'Session' es el área de trabajo donde se guardan, consultan y modifican objetos.
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def get_db():
    """Función generadora para obtener y cerrar la sesión de la base de datos."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

```python
# mi_escuela/escuela.py (CÓDIGO ORM)

from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from .database import Base

class Escuela(Base):
    """
    Clase que mapea la tabla 'escuelas' para la persistencia de datos.
    Ya no almacena profesores y alumnos en memoria, solo define la estructura
    de la tabla y sus relaciones.
    """
    # Mapeo a la tabla 'escuelas'
    __tablename__ = 'escuelas'

    # Columnas:
    id = Column(Integer, primary_key=True, index=True)
    nombre = Column(String, nullable=False)
    localizacion = Column(String, nullable=False) # Ahora se llama "localizacion" en lugar de "localidad"
    director = Column(String, nullable=False)     # Ahora se llama "director" en lugar de "responsable"

    # Relación: Una Escuela puede tener múltiples Profesores
```

```
    # El Profesor contendrá la Clave Foránea a esta tabla (escuela_id).
    profesores = relationship("Profesor", back_populates="escuela_asociada")

    def __init__(self, nombre: str, localizacion: str, director: str):
        '''
        Inicializa una nueva instancia de Escuela.
        '''
        # constructor simple para crear objetos en Python
        self.nombre = nombre
        self.localizacion = localizacion
        self.director = director

    def __str__(self):
        '''
            Representación en cadena del objeto Escuela.
        '''
        return f"Escuela: {self.nombre} (ID: {self.id}), Director: {self.director}, Localidad: {self.localiza-
cion}"
```

```
# mi_escuela/profesor.py

from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship

# Importamos la Base del ORM
from .database import Base

class Profesor(Base):
    """
        Define el mapeo a la tabla 'profesores'.
        Un profesor pertenece a una Escuela (ForeignKey).
        Un profesor tiene muchos Alumnos (relationship).
    """
    __tablename__ = 'profesores'

    # Columnas de la tabla:
    id = Column(Integer, primary_key=True, index=True)
```

```
dni = Column(String, unique=True, index=True, nullable=False)
nombre = Column(String, nullable=False)
tipo = Column(String, nullable=False)

# CLAVE FORÁNEA a la tabla ESCUELAS: Mucho a Uno
# Cada profesor DEBE estar asociado a un ID de escuela válido.
escuela_id = Column(Integer, ForeignKey('escuelas.id'), nullable=False)

# Relaciones ORM:
# Relación con la clase Escuela (Muchos Profesores a Una Escuela)
escuela_asociada = relationship("Escuela", back_populates="profesores")

# Relación con la clase Alumno (Un Profesor a Muchos Alumnos)
alumnos_a_cargo = relationship("Alumno", back_populates="profesor_responsable")


# El constructor se simplifica para crear objetos ORM
def __init__(self, nombre: str, dni: str, tipo: str, escuela_asociada: object = None):
    self.nombre = nombre
    self.dni = dni.upper()
    self.tipo = tipo.capitalize()
    if escuela_asociada:
        # Asigna la relación
        self.escuela_asociada = escuela_asociada


def __str__(self):
    # Muestra el nombre de la escuela a través de la relación
    escuela_nombre = self.escuela_asociada.nombre if hasattr(self, 'escuela_asociada') else "N/A"
    return f"Profesor: {self.nombre} (DNI: {self.dni}), Tipo: {self.tipo}, Escuela: {escuela_nombre}"
```

```python
# mi_escuela/alumno.py

from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship
from .database import Base


class Alumno(Base):
    """
        Define el mapeo a la tabla 'alumnos'.
        Un alumno pertenece a un Profesor (ForeignKey).
    """
    __tablename__ = 'alumnos'

    # Columnas de la tabla:
    id = Column(Integer, primary_key=True, index=True)
    # NIA del alumno
    nia = Column(String, unique=True, index=True, nullable=False)
    nombre = Column(String, nullable=False)
    curso = Column(String, nullable=False)

    # CLAVE FORÁNEA a la tabla PROFESORES: Mucho a Uno
    # Cada alumno DEBE estar asociado a un ID de profesor válido.
    profesor_id = Column(Integer, ForeignKey('profesores.id'), nullable=False)

    # Relación ORM:
    # Relación con la clase Profesor (Muchos Alumnos a Un Profesor)
    # Esto permite acceder al objeto Profesor desde el objeto Alumno.
    profesor_responsable = relationship("Profesor", back_populates="alumnos_a_cargo")


    # El constructor se simplifica para crear objetos ORM
    def __init__(self, nombre: str, nia: str, curso: str, profesor_responsable: object = None):
        self.nombre = nombre
        self.nia = dni_nia.upper()
        self.curso = curso
        if profesor_responsable:
            # Asigna la relación
```

```
        self.profesor_responsable = profesor_responsable


    def __str__(self):
        # Muestra el nombre del profesor responsable a través de la relación
        resp_nombre = self.profesor_responsable.nombre if hasattr(self, 'profesor_responsable') else
"N/A"
        return f"Alumno: {self.nombre} (ID: {self.id}), NIA: {self.nia}, Curso: {self.curso} (Resp: {resp_nom-
bre})"
```

```
# main.py
import sys
import os

# Agrega la ruta del directorio actual (Proyecto_SQLAlchemy) al sys.path de Python.
# Esto asegura que pueda encontrar la carpeta 'mi_escuela' como un paquete.

current_dir = os.path.dirname(os.path.abspath(__file__))
if current_dir not in sys.path:
    sys.path.append(current_dir)

# Importamos las herramientas de la base de datos y los modelos
from mi_escuela.database import Base, engine, SessionLocal
from mi_escuela.escuela import Escuela
from mi_escuela.profesor import Profesor
from mi_escuela.alumno import Alumno
from typing import Optional, List

# ============================================================================
# FUNCIONES DE AYUDA PARA LA GESTIÓN DE LA BD
# ============================================================================

def crear_tablas():
    """Crea todas las tablas definidas por los modelos si no existen."""
    Base.metadata.create_all(bind=engine)
```

```
def get_escuela_actual(db) -> Optional[Escuela]:
    """
        Recupera la primera escuela registrada para usarla en la gestión
        Args:
            db: Sesión activa de la base de datos
        Returns:
            Escuela: La primera escuela encontrada o None si no hay ninguna
    """
    escuela = db.query(Escuela).first()
    if not escuela:
        print("ADVERTENCIA: No se encontró ninguna escuela. Cree una primero.")
    return escuela

def get_profesor_por_dni(db, dni: str) -> Optional[Profesor]:
    """
        Busca un profesor por DNI.
        Args:
            db: Sesión activa de la base de datos
            dni (str): DNI del profesor a buscar
        Returns:
            Profesor: El objeto Profesor encontrado o None si no existe
    """
    return db.query(Profesor).filter(Profesor.dni == dni.upper()).first()

# ============================================================================
# LÓGICA DE GESTIÓN DE PROFESORES (CRUD)
#
# ============================================================================

# CREATE (Crear) -
def agregar_profesor(db, escuela_actual: Escuela):
    """
        Agrega un profesor.
        Args:
            db: Sesión activa de la base de datos
            escuela_actual (Escuela): La escuela a la que se asociará el profesor
        Returns:
            None
```

```
    """
    print("\n--- Agregar Profesor ---")
    nombre = input("Nombre del profesor: ")
    dni = input("DNI del profesor: ").upper()
    tipo = input("Tipo (Ciencias/Letras/Mixto): ").capitalize()

    if get_profesor_por_dni(db, dni):
        print(f"ERROR: El profesor con DNI {dni} ya existe.")
        return

    nuevo_profesor = Profesor(nombre=nombre, dni=dni, tipo=tipo, escuela_asociada=escuela_actual)

    try:
        db.add(nuevo_profesor)
        db.commit()
        print(f"INFO: Profesor {nombre} agregado con éxito a la base de datos.")
    except Exception as e:
        db.rollback()
        print(f"ERROR al guardar el profesor: {e}")

# READ (Leer) -
def mostrar_profesores(db, escuela_actual: Escuela):
    """
        Muestra todos los profesores de la escuela.
        Args:
            db: Sesión activa de la base de datos
            escuela_actual (Escuela): La escuela cuyos profesores se mostrarán
        Returns:
            None
    """
    print("\n--- Listado de Profesores ---")
    profesores: List[Profesor] = db.query(Profesor).filter(
        Profesor.escuela_id == escuela_actual.id
    ).all()

    if not profesores:
        print("No hay profesores registrados en esta escuela.")
        return
```

```
    for p in profesores:
        print(p)

# UPDATE (Actualizar) -
def modificar_profesor(db):
    """
        Modifica los atributos de un profesor por DNI.
        Args:
            db: Sesión activa de la base de datos
            Returns:
            None
    """
    print("\n--- Modificar Profesor ---")
    dni = input("DNI del profesor a modificar: ").upper()
    profesor = get_profesor_por_dni(db, dni)

    if not profesor:
        print(f"ERROR: Profesor con DNI {dni} no encontrado.")
        return

    try:
        print(f"Modificando a: {profesor.nombre} (Deje en blanco para no modificar)")

        nuevo_nombre = input(f"Nuevo nombre ({profesor.nombre}): ")
        if nuevo_nombre:
            profesor.nombre = nuevo_nombre

        nuevo_tipo = input(f"Nuevo tipo ({profesor.tipo}): ").capitalize()
        if nuevo_tipo:
            profesor.tipo = nuevo_tipo

        db.commit() # La sesión rastrea los cambios y los guarda
        print(f"INFO: Profesor {dni} modificado con éxito.")

    except Exception as e:
        db.rollback()
        print(f"ERROR al modificar el profesor: {e}")
```

```
# DELETE (Eliminar) -
def eliminar_profesor(db):
    """
        Elimina un profesor por DNI y los alumnos asociados (si la BD lo permite).
        Args:
            db: Sesión activa de la base de datos
        Returns:
            None
    """
    print("\n--- Eliminar Profesor ---")
    dni = input("DNI del profesor a eliminar: ").upper()
    profesor = get_profesor_por_dni(db, dni)

    if not profesor:
        print(f"ERROR: Profesor con DNI {dni} no encontrado.")
        return

    # Comprobar si tiene alumnos.
    if profesor.alumnos_a_cargo:
        print(f"ADVERTENCIA: El profesor tiene {len(profesor.alumnos_a_cargo)} alumnos asignados.")
        confirm = input("¿Confirmar eliminación? (S/N): ").upper()
        if confirm != 'S':
            print("Operación cancelada.")
            return

    try:
        db.delete(profesor)
        db.commit()
        print(f"INFO: Profesor {dni} eliminado con éxito.")
    except Exception as e:
        db.rollback()
        print(f"ERROR al eliminar el profesor: Asegúrese de que no tenga alumnos asignados si hay restricciones de FK. {e}")


def menu_profesores():
    """Submenú de gestión de profesores."""
```

```
    db = SessionLocal()
    try:
        escuela = get_escuela_actual(db)
        if not escuela: return

        while True:
            print("\n--- Menú de Profesores ---")
            print("1. Agregar Profesor")
            print("2. Mostrar Profesores")
            print("3. Modificar Profesor")
            print("4. Eliminar Profesor")
            print("0. Volver al Menú Principal")
            opcion = input("Seleccione una opción: ")

            if opcion == '1':
                agregar_profesor(db, escuela)
            elif opcion == '2':
                mostrar_profesores(db, escuela)
            elif opcion == '3':
                modificar_profesor(db)
            elif opcion == '4':
                eliminar_profesor(db)
            elif opcion == '0':
                break
            else:
                print("Opción no válida.")
    finally:
        db.close()

# ==============================================================================
# LÓGICA DE GESTIÓN DE ALUMNOS (CRUD)
# ==============================================================================

# Helper para buscar alumno por NIA
def get_alumno_por_nia(db, nia: str) -> Optional[Alumno]:
    """
        Busca un alumno por NIA.
        Args:
```

```
        db: Sesión activa de la base de datos
        nia (str): NIA del alumno a buscar
        Returns:
        Alumno: El objeto Alumno encontrado o None si no existe
    """
    return db.query(Alumno).filter(Alumno.nia == nia.upper()).first()

# CREATE (Crear) -
def agregar_alumno(db, escuela_actual: Escuela):
    """
        Agrega un alumno, asociándolo a un profesor.
        Args:
            db: Sesión activa de la base de datos
            escuela_actual (Escuela): La escuela a la que se asociará el alumno
        Returns:
            None
    """
    print("\n--- Agregar Alumno ---")
    nombre = input("Nombre del alumno: ")
    nia = input("NIA del alumno: ").upper()
    curso = input("Curso: ")
    dni_profesor = input("DNI del profesor responsable: ").upper()

    if get_alumno_por_nia(db, nia):
        print(f"ERROR: El alumno con NIA {nia} ya existe.")
        return

    profesor_responsable = get_profesor_por_dni(db, dni_profesor)
    if not profesor_responsable:
        print(f"ERROR: Profesor con DNI {dni_profesor} no encontrado.")
        return

    nuevo_alumno = Alumno(
        nombre=nombre,
        nia=nia,
        curso=curso,
        profesor_responsable=profesor_responsable
    )
```

```
    try:
        db.add(nuevo_alumno)
        db.commit()
        print(f"INFO: Alumno {nombre} agregado con éxito.")
    except Exception as e:
        db.rollback()
        print(f"ERROR al guardar el alumno: {e}")

# READ (Leer) -
def mostrar_alumnos(db, escuela_actual: Escuela):
    """
        Muestra todos los alumnos de la escuela.
        Args:
            db: Sesión activa de la base de datos
            escuela_actual (Escuela): La escuela cuyos alumnos se mostrarán
        Returns:
            None
    """
    print("\n--- Listado de Alumnos ---")

    alumnos: List[Alumno] = db.query(Alumno).join(Profesor).filter(
        Profesor.escuela_id == escuela_actual.id
    ).all()

    if not alumnos:
        print("No hay alumnos registrados en esta escuela.")
        return

    for a in alumnos:
        print(a)

# UPDATE (Actualizar) - ¡NUEVO!
def modificar_alumno(db):
    """
        Modifica los atributos de un alumno por NIA.
        Args:
            db: Sesión activa de la base de datos
```

```
        Returns:
            None
    """
    print("\n--- Modificar Alumno ---")
    nia = input("NIA del alumno a modificar: ").upper()
    alumno = get_alumno_por_nia(db, nia)

    if not alumno:
        print(f"ERROR: Alumno con NIA {nia} no encontrado.")
        return

    try:
        print(f"Modificando a: {alumno.nombre} (Deje en blanco para no modificar)")

        nuevo_nombre = input(f"Nuevo nombre ({alumno.nombre}): ")
        if nuevo_nombre:
            alumno.nombre = nuevo_nombre

        nuevo_curso = input(f"Nuevo curso ({alumno.curso}): ")
        if nuevo_curso:
            alumno.curso = nuevo_curso

        nuevo_profesor_dni = input(f"Nuevo DNI del responsable ({alumno.profesor_responsable.dni}):
").upper()
        if nuevo_profesor_dni:
            nuevo_profesor = get_profesor_por_dni(db, nuevo_profesor_dni)
            if nuevo_profesor:
                alumno.profesor_responsable = nuevo_profesor # Asigna la nueva relación
                print(f"INFO: Responsable cambiado a {nuevo_profesor.nombre}.")
            else:
                print(f"ADVERTENCIA: Profesor con DNI {nuevo_profesor_dni} no encontrado. Responsable
no modificado.")

        db.commit()
        print(f"INFO: Alumno {nia} modificado con éxito.")

    except Exception as e:
        db.rollback()
```

```
        print(f"ERROR al modificar el alumno: {e}")

# DELETE (Eliminar) -
def eliminar_alumno(db):
    """ORM: Elimina un alumno por NIA."""
    print("\n--- Eliminar Alumno ---")
    nia = input("NIA del alumno a eliminar: ").upper()
    alumno = get_alumno_por_nia(db, nia)

    if not alumno:
        print(f"ERROR: Alumno con NIA {nia} no encontrado.")
        return

    try:
        db.delete(alumno)
        db.commit()
        print(f"INFO: Alumno {nia} eliminado con éxito.")
    except Exception as e:
        db.rollback()
        print(f"ERROR al eliminar el alumno: {e}")

def menu_alumnos():
    """Submenú de gestión de alumnos."""
    db = SessionLocal()
    try:
        escuela = get_escuela_actual(db)
        if not escuela: return

        while True:
            print("\n--- Menú de Alumnos ---")
            print("1. Agregar Alumno")
            print("2. Mostrar Alumnos")
            print("3. Modificar Alumno")
            print("4. Eliminar Alumno")
            print("0. Volver al Menú Principal")
            opcion = input("Seleccione una opción: ")

            if opcion == '1':
```

```
            mostrar_profesores(db, escuela)
            agregar_alumno(db, escuela)
        elif opcion == '2':
            mostrar_alumnos(db, escuela)
        elif opcion == '3':
            modificar_alumno(db, escuela)
        elif opcion == '4':
            eliminar_alumno(db, escuela)
        elif opcion == '0':
            break
        else:
            print("Opción no válida.")
    finally:
        db.close()

# ========================================================================
# MENÚ PRINCIPAL Y PUNTO DE INICIO
# ========================================================================

def crear_escuela_inicial(db):
    """Crea una escuela si no existe ninguna (necesaria para asociar profesores)."""
    if db.query(Escuela).first():
        return

    print("\n--- Primera Ejecución: Creación de Escuela ---")
    print("Se requiere crear una escuela para empezar la gestión.")
    nombre = input("Nombre de la escuela: ")
    localizacion = input("Localidad: ")
    director = input("Director responsable: ")

    nueva_escuela = Escuela(nombre=nombre, localizacion=localizacion, director=director)

    try:
        db.add(nueva_escuela)
        db.commit()
        print(f"INFO: Escuela '{nombre}' creada con éxito.")
    except Exception as e:
        db.rollback()
```

```
        print(f"ERROR al crear la escuela: {e}")

def menu_principal():
    """El loop principal del programa."""

    #Configuración inicial
    crear_tablas()

    # Abrir sesión para chequear/crear la escuela inicial
    db_setup = SessionLocal()
    try:
        crear_escuela_inicial(db_setup)
    finally:
        db_setup.close()

    while True:
        print("\n=== Sistema de Gestión Escolar ===")
        print("1. Gestión de Profesores")
        print("2. Gestión de Alumnos")
        print("0. Salir")
        opcion = input("Seleccione una opción: ")

        if opcion == '1':
            menu_profesores()
        elif opcion == '2':
            menu_alumnos()
        elif opcion == '0':
            print("Saliendo del programa. ¡Datos guardados en escuela.db!")
            break
        else:
            print("Opción no válida.")


if __name__ == "__main__":
    menu_principal()
```

Por consola :

```
(venv) C:\Users\miria\Documents\Proyecto_SQLAlchemy>python main.py
06,602 INFO sqlalchemy.engine.Engine BEGIN (implicit)
06,602 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("escuelas")
06,602 INFO sqlalchemy.engine.Engine [raw sql] ()
 10:06,606 INFO sqlalchemy.engine.Engine PRAGMA temp.table_info("escuelas")
 10:06,606 INFO sqlalchemy.engine.Engine [raw sql] ()
 10:06,606 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("profesores")
 10:06,606 INFO sqlalchemy.engine.Engine [raw sql] ()
 10:06,606 INFO sqlalchemy.engine.Engine PRAGMA temp.table_info("profesores")
 10:06,606 INFO sqlalchemy.engine.Engine [raw sql] ()
 10:06,606 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("alumnos")
 10:06,607 INFO sqlalchemy.engine.Engine [raw sql] ()
 10:06,607 INFO sqlalchemy.engine.Engine PRAGMA temp.table_info("alumnos")
 10:06,607 INFO sqlalchemy.engine.Engine [raw sql] ()
 10:06,607 INFO sqlalchemy.engine.Engine
CREATE TABLE escuelas (
        id INTEGER NOT NULL,
        nombre VARCHAR NOT NULL,
        localizacion VARCHAR NOT NULL,
        director VARCHAR NOT NULL,
        PRIMARY KEY (id)
)


 10:06,608 INFO sqlalchemy.engine.Engine [no key 0.00012s] ()
 10:06,616 INFO sqlalchemy.engine.Engine CREATE INDEX ix_escuelas_id ON escuelas (id)
 10:06,617 INFO sqlalchemy.engine.Engine [no key 0.00019s] ()
 10:06,622 INFO sqlalchemy.engine.Engine
```





```
CREATE TABLE profesores (
	id INTEGER NOT NULL,
	dni VARCHAR NOT NULL,
	nombre VARCHAR NOT NULL,
	tipo VARCHAR NOT NULL,
	escuela_id INTEGER NOT NULL,
	PRIMARY KEY (id),
	FOREIGN KEY(escuela_id) REFERENCES escuelas (id)
)


10:06,622 INFO sqlalchemy.engine.Engine [no key 0.00030s] ()
10:06,627 INFO sqlalchemy.engine.Engine CREATE UNIQUE INDEX ix_profesores_dni ON profesores (dni)
10:06,627 INFO sqlalchemy.engine.Engine [no key 0.00019s] ()
10:06,632 INFO sqlalchemy.engine.Engine CREATE INDEX ix_profesores_id ON profesores (id)
10:06,632 INFO sqlalchemy.engine.Engine [no key 0.00022s] ()
10:06,638 INFO sqlalchemy.engine.Engine
CREATE TABLE alumnos (
	id INTEGER NOT NULL,
	nia VARCHAR NOT NULL,
	nombre VARCHAR NOT NULL,
	curso VARCHAR NOT NULL,
	profesor_id INTEGER NOT NULL,
	PRIMARY KEY (id),
	FOREIGN KEY(profesor_id) REFERENCES profesores (id)
)
```

10:06,639 INFO sqlalchemy.engine.Engine [no key 0.00035s] ()

10:06,645 INFO sqlalchemy.engine.Engine CREATE INDEX ix_alumnos_id ON alumnos (id)

10:06,646 INFO sqlalchemy.engine.Engine [no key 0.00019s] ()

10:06,652 INFO sqlalchemy.engine.Engine CREATE UNIQUE INDEX ix_alumnos_nia ON alumnos (nia)

10:06,652 INFO sqlalchemy.engine.Engine [no key 0.00018s] ()

10:06,657 INFO sqlalchemy.engine.Engine COMMIT

10:06,664 INFO sqlalchemy.engine.Engine BEGIN (implicit)

10:06,668 INFO sqlalchemy.engine.Engine SELECT escuelas.id AS escuelas_id, escuelas.nombre AS escuelas_nombre, escuelas.localizacion AS escuelas_localizacion, escuelas.director AS escuelas_director

FROM escuelas

LIMIT ? OFFSET ?

10:06,668 INFO sqlalchemy.engine.Engine [generated in 0.00030s] (1, 0)

--- Primera Ejecución: Creación de Escuela ---

Se requiere crear una escuela para empezar la gestión.

Nombre de la escuela:

PREGUNTA ENTREVISTA 07

Realiza un programa que recibirá por parámetros de consola dos nombres : el del archivo de entrada y el del archivo de salida.
Además, para tener este programa en un ejecutable y poder distribuirlo deberemos empacar el programa en un ejecutable utilizando la biblioteca https://www.pyinstaller.org/.
El programa implementará dos funciones: “esPalindromo” y “esPrimo”
Importante: estas funciones tendrán que estar comprobadas con un test unitario. Recomendamos por este caso "doctest" **https://docs.python.org/es/3/library/doctest.html**
pero vale cualquier otra utilidad para realizar tests unitarios.
El programa debe leer el archivo de entrada. Este archivo contiene un número por líneas (no se especifica cuántas líneas, debe funcionar por cualquier número).

Nuestro programa debe imprimir en el fichero de salida dos líneas y después un listado con un número por línea:

- La primera línea debe decir cuántos números eran “palíndromos” (capicúa)
- Otra línea diciendo cuántos eran “primos”.
- Por último, “n” líneas indicando qué “n” números eran al mismo tiempo primos y palíndromos.

Por ejemplo:
Archivo “ejemploEntrada.txt”

5

7

11

12

14

14

22

Archivo “ejemplo Salida.txt”
Hay 4 números palíndromos

Hay 4 números primos.

5

7

11

Consejo 1: este tutorial puede ayudarte con "Doctest"

https://www.digitalocean.com/community/tutorials/how-to-write-doctests-in-python

Consejo 2: este tutorial puede ayudarte con "PyInstaller"

https://www.devdungeon.com/content/pyinstaller-tutorial

doctest es una biblioteca estándar de Python que permite escribir ejemplos de uso dentro de los comentarios de una función. Luego, Python puede ejecutar esos ejemplos automáticamente para verificar que la función funciona correctamente.

Ejemplo:

>>> esPalindromo(121) → Esto simula que el usuario escribe ese comando en consola.

True → Es el resultado esperado.

Al ejecutar doctest, Python compara el resultado real con el esperado. Si no coinciden, te avisa.

Para comprobar si instalación:

terminal o consola y ejecutar estos comandos:

PyInstaller --version

Instala PyInstaller

pip install pyinstaller

Pregunta07/

```
|
├─── es_palindromo.py
├─── es_primo.py
├─── main.py
├─── ejemploEntrada.txt
```

```python
# es_palindromo.py
def esPalindromo(numero):
    """
        Devuelve True si el número es palíndromo (capicúa).
        Args:
            n (int): Número a comprobar.
        Returns:
            bool: True si el número es palíndromo, False en caso contrario.

        >>> esPalindromo(121)
        True
        >>> esPalindromo(123)
        False
        >>> esPalindromo(11)
        True
        >>> esPalindromo(5)
        True
        >>> esPalindromo(1221)
        True
    """
    # Convertimos el número a cadena (string)
    cadena_numero = str(numero)
    # Comparamos la cadena original con su inversa
    return cadena_numero == cadena_numero[::-1]

if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

```python
#esPrimo.py
import math

def esPrimo(numero):
    """
        Comprueba si un número entero es primo.

        Un número primo es un número natural mayor que 1 que tiene
        únicamente dos divisores distintos y positivos: él mismo y el 1.
        Args:
            n (int): Número a comprobar.
        Returns:
            bool: True si el número es primo, False en caso contrario.

        >>> esPrimo(7)
        True
        >>> esPrimo(1)
        False
        >>> esPrimo(4)
        False
        >>> esPrimo(11)
        True
        >>> esPrimo(12)
        False
        >>> esPrimo(2)
        True
    """
    if numero <= 1:
        return False
    #  divisores desde 2 hasta la raíz cuadrada del número
    for i in range(2, int(math.sqrt(numero)) + 1):
        if numero % i == 0:
            return False
    return True

if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

```python
# main.py

import sys
from es_palindromo import esPalindromo
from es_primo import esPrimo

def main():
    if len(sys.argv) != 3:
        print("Uso: python main.py <archivo_entrada> <archivo_salida>")
        sys.exit(1)

    archivo_entrada = sys.argv[1]
    archivo_salida = sys.argv[2]

    with open(archivo_entrada, 'r', encoding='utf-8') as f:
        numeros = [int(line.strip()) for line in f if line.strip().isdigit()]

    palindromos = [n for n in numeros if esPalindromo(n)]
    primos = [n for n in numeros if esPrimo(n)]
    ambos = [n for n in numeros if esPalindromo(n) and esPrimo(n)]

    with open(archivo_salida, 'w', encoding='utf-8') as f:
        f.write(f"Hay {len(palindromos)} números palíndromos\n")
        f.write(f"Hay {len(primos)} números primos.\n")
        for n in ambos:
            f.write(f"{n}\n")

if __name__ == "__main__":
    main()
```

#ejemploEntrada.py

5

7

11

12

14

14

22

Antes de empaquetar, asegurarse de que todo funciona:

Probar los tests de cada función

python es_palindromo.py

python es_primo.py

#ejemploSalida.txt

Hay 4 números palíndromos

Hay 3 números primos.

5

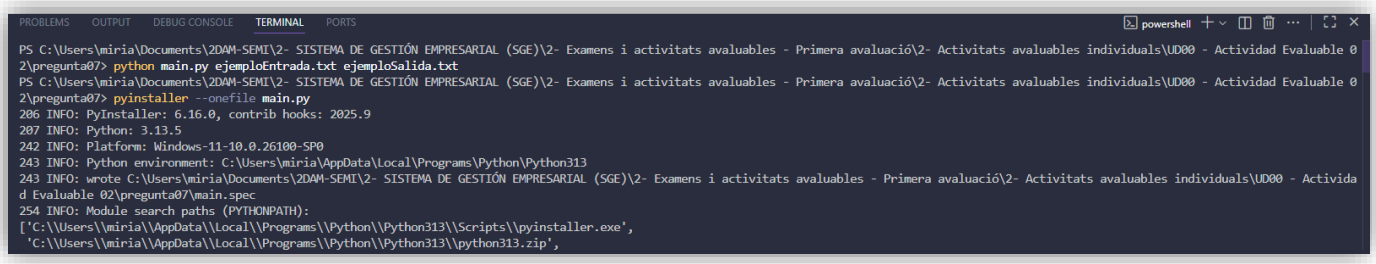
7

11

Empaquetar con PyInstaller

Desde la carpeta donde está main.py, ejecutar en la consola:

pyinstaller --onefile main.py



Esto generará un ejecutable en la carpeta dist/llamado:

- main.exe(en Windows)
- main (en Linux/macOS)

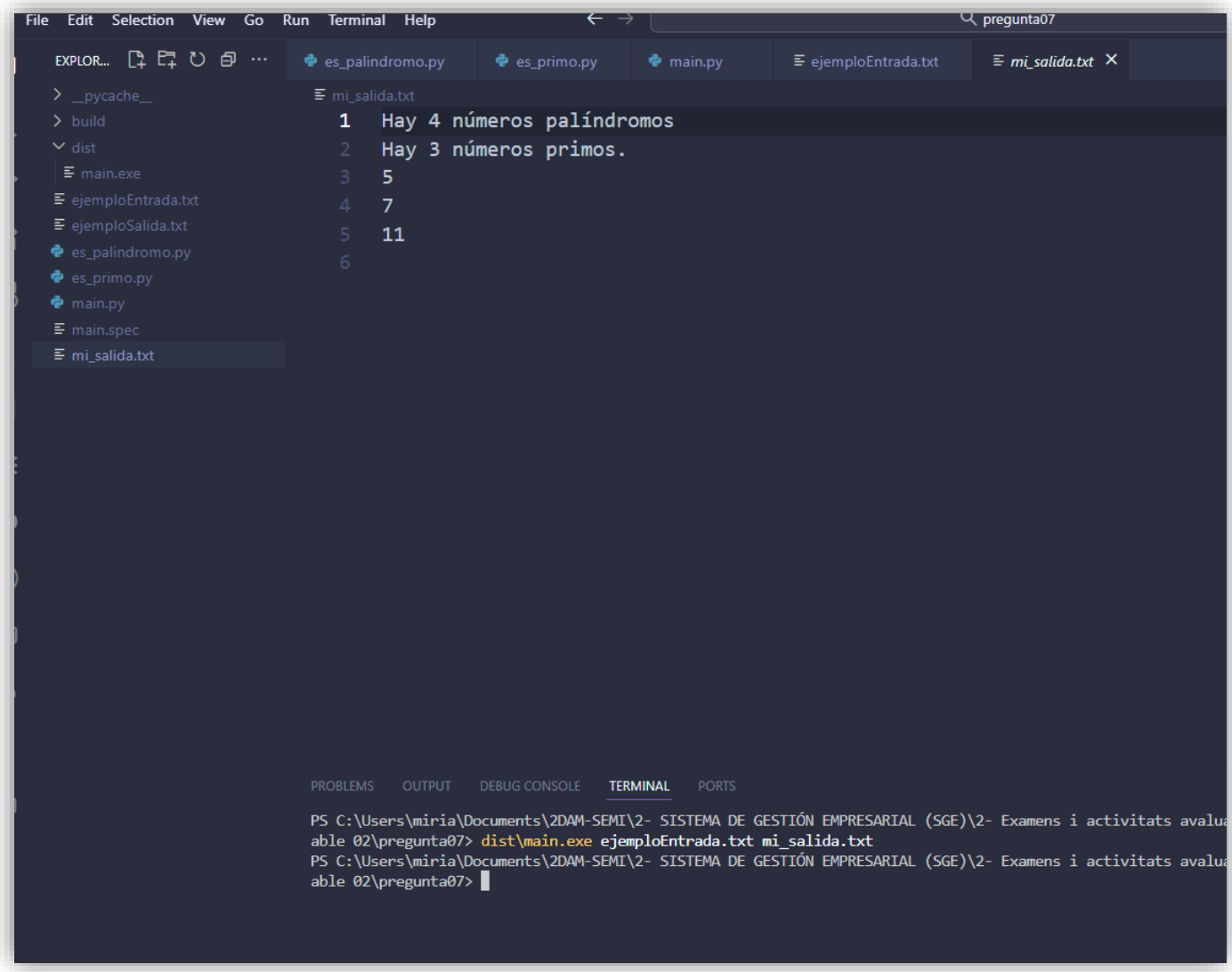
PyInstaller empaqueta el código fuente tal como está en ese momento dentro del ejecutable. Si cambias algo después, hay que volver a ejecutarlo.

Ejecutar el ejecutable

./dist/main ejemploEntrada.txt ejemploSalida.txt

En Windows:

dist\main.exe ejemploEntrada.txt mi_salida.txt


File Edit Selection View Go Run Terminal Help
pregunta07
EXPLOR...
es_palindromo.py
es_primo.py
main.py
ejemploEntrada.txt
mi_salida.txt
__pycache__
build
dist
main.exe
ejemploEntrada.txt
ejemploSalida.txt
es_palindromo.py
es_primo.py
main.py
main.spec
mi_salida.txt
mi_salida.txt
1 Hay 4 números palíndromos
2 Hay 3 números primos.
3 5
4 7
5 11
6
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\miria\Documents\2DAM-SEMI\2- SISTEMA DE GESTIÓN EMPRESARIAL (SGE)\2- Examens i activitats avalua
able 02\pregunta07> dist\main.exe ejemploEntrada.txt mi_salida.txt
PS C:\Users\miria\Documents\2DAM-SEMI\2- SISTEMA DE GESTIÓN EMPRESARIAL (SGE)\2- Examens i activitats avalua
able 02\pregunta07>

PREGUNTA ENTREVISTA 08

Crea un programa Python 3 que nos ayude a organizar los archivos de la carpeta actual. El programa tendrá una lista con extensiones de programas (debe funcionar con cualquier-ra).

El programa al ejecutarse, por cada extensión, moverá todos los archivos con esa extensión a una carpeta con ese nombre.Por ejemplo, con lista=[“png”,”mp4”, “doc”], el programa move-ría todos los archivos con extensión “.png” de la carpeta actual a una carpeta llamada “png” Lo mismo con los archivos “.mp4” en la carpeta “mp4” y los archivos “.doc” con la carpeta “doc”.

Además, para tener este programa en un ejecutable y poder distribuirlo deberemos empacar el programa en un ejecutable utilizando la biblioteca **https://www.pyinstaller.org/**

Consejo 01: este tutorial y cheatsheet pueden ser útiles para solucionar este problema

https://unipython.com/operaciones-con-archivos-y-carpetas-en-python/

https://github.com/sergarb1/CheatSheetsCastellano/raw/master/Cheatsheet%20%20Python%20para%20crear%20Scripts%20de%20Sistemas%20operativos.pdf

Consejo 02: este tutorial puede ayudarte con “PyInstaller”

https://www.devdungeon.com/content/pyinstaller-tutorial

PREGUNTA ENTREVISTA 09

Utilizando la biblioteca **https://pypi.org/project/python-barcode/ ,** debemos leer un archivo ".csv" pasado por parámetros de consola.

Este archivo tendrá un formato en el que cada línea usará "nombrealumno,ID".

Por ejemplo

"Alumno 01","1"

"Alumno 02","2"

Deberemos realizar un programa, que leyendo ese fichero ".csv" genere por cada alumno genere un fichero "nomalumne.png" con un código de barras en formato "EAN 13" que tenga como información el "ID" del alumno.

Consejo 01: para leer archivos ".csv" puedes utilizar la biblioteca "csv" o la biblioteca "pandas". Aquí tienes ejemplos:

https://code.tutsplus.com/es/tutorials/how-to-read-and-write-csv-files-in-python--cms-29907

https://pythonbros.com/leer-csv-con-pandas/

Consejo 02: puede instalar Python Barcode con '**pip3 install python-barcode**' y '**pip3 install "Python-barcode[images]"**'.

Consejo 03: tienes ejemplos de funcionamiento de Python Barcode que son útiles directamente en

https://pypi.org/project/python-barcode/.

Recuerdos comprender que estás haciendo. Tienes que tener claro que es EAN13 antes de codificar:

https://es.wikipedia.org/wiki/European_Article_Number

- Instalar la biblioteca necesaria:
  pip install python-barcode
  pip install Pillow # permitirá usar ImageWriter para generar códigos de barras como imágenes PNG.

```
s\UD00 - Actividad Evaluable 02\pregunta9> pip install Pillow
Collecting Pillow
  Downloading pillow-12.0.0-cp313-cp313-win_amd64.whl.metadata (9.0 kB)
Downloading pillow-12.0.0-cp313-cp313-win_amd64.whl (7.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.0/7.0 MB 13.9 MB/s eta 0:00:00
Installing collected packages: Pillow
Successfully installed Pillow-12.0.0

[notice] A new release of pip is available: 25.1.1 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\miria\Documents\2DAM-SEMI\2- SISTEMA DE GESTIÓN EMPRESARIAL (SGE)\2-
s\UD00 - Actividad Evaluable 02\pregunta9>
```

- Código del Programa: generar_codigos.py

```python
# generar_codigos.py

import sys
import csv
import os
from barcode import EAN13
from barcode.writer import ImageWriter

# --- 1. Definición del Formato EAN-13 ---
# EAN-13 necesita un número de 12 dígitos, ya que el dígito 13 (el de control)
# es calculado automáticamente por la librería.

def formatear_id_para_ean13(id_alumno):
    """
    Rellena el ID del alumno con ceros a la izquierda para alcanzar 12 dígitos.
    Args:
        id_alumno (str o int): El ID del alumno.
    Returns:
        str: El ID formateado con 12 dígitos.
```

```
    """
    # Se asegura de que el ID sea una cadena
    str_id = str(id_alumno)
    # Rellena con ceros a la izquierda hasta 12 dígitos
    return str_id.zfill(12)

# --- 2. Lectura de Argumentos y CSV ---

def generar_codigos_desde_csv(ruta_csv):
    """
    Lee el archivo CSV y genera un código de barras EAN-13 para cada alumno.
    Args:
        ruta_csv (str): La ruta al archivo CSV con los datos de los alumnos.
    Returns:
        None
    """
    print(f"Procesando el archivo CSV: {ruta_csv}")

    try:
        with open(ruta_csv, 'r', newline='', encoding='utf-8') as archivo_csv:
            # csv.reader, se asume que el separador es la coma (",")
            # y que los campos pueden estar entre comillas dobles (")
            lector_csv = csv.reader(archivo_csv, delimiter=',', quotechar='"')

            # Iterar sobre cada fila del archivo CSV
            for i, fila in enumerate(lector_csv):
                # Omitir filas vacías
                if not fila:
                    continue

                # Verificar el formato de la fila
                if len(fila) < 2:
                    print(f"Advertencia: Fila {i+1} ignorada. Formato incorrecto: {fila}")
                    continue

                # Desempaquetar los datos: nombre_alumno, ID
                nombre_alumno_crudo = fila[0].strip()
                id_alumno_crudo = fila[1].strip()
```

```
            # Limpieza y preparación del nombre
            # Se eliminan caracteres no válidos para nombres de archivo
            nombre_archivo_base = nombre_alumno_crudo.replace(" ", "_").replace("\"", "").replace("'", "")

            # Preparar ID para EAN-13
            try:
                codigo_ean = formatear_id_para_ean13(id_alumno_crudo)
            except ValueError:
                print(f"Error en fila {i+1}: El ID '{id_alumno_crudo}' no es un número válido. Saltando.")
                continue

            # --- 3. Generación del Código de Barras ---

            # Crear el objeto EAN13. Se usa ImageWriter para generar PNGs.
            try:
                ean = EAN13(codigo_ean, writer=ImageWriter())

                # Definir el nombre del archivo de salida: "nomalumne.png"
                nombre_archivo_salida = f"{nombre_archivo_base}.png"

                # Guardar la imagen del código de barras
                nombre_guardado = ean.save(nombre_archivo_salida)
                print(f"Generado: '{nombre_guardado}' para el alumno '{nombre_alumno_crudo}' con ID:
{id_alumno_crudo}")

            except Exception as e:
                print(f"Error al generar el código de barras para '{nombre_alumno_crudo}' (ID:
{id_alumno_crudo}): {e}. Saltando.")

    except FileNotFoundError:
        print(f"Error: El archivo '{ruta_csv}' no se encontró.")
        sys.exit(1)
    except Exception as e:
        print(f"Un error inesperado ocurrió: {e}")
        sys.exit(1)
```

```python
if __name__ == "__main__":
    # sys.argv[0] es el nombre del script
    if len(sys.argv) < 2:
        print("Uso: python generar_codigos.py <ruta_del_archivo.csv>")
        sys.exit(1)

    # sys.argv[1] es el primer argumento pasado por consola (la ruta del archivo)
    archivo_entrada = sys.argv[1]
    generar_codigos_desde_csv(archivo_entrada)
```

Componente	Descripción
sys.argv	Lista que contiene los argumentos de línea de comandos. sys.argv[1] contiene la ruta del archivo CSV.
csv.reader	Se utiliza para leer el archivo CSV fila por fila. Se configura para manejar el delimitador de coma y el carácter de cita doble.
formatear_id_para_ean13	Función crucial que utiliza .zfill(12) para asegurar que el ID de alumno tenga exactamente 12 dígitos, un requisito para el formato EAN-13 antes de que la biblioteca añada el dígito de control número 13.
EAN13(...)	Genera el objeto código de barras.
writer=ImageWriter()	Indica a la biblioteca python-barcode que debe usar Pillow (instalado con [image]) para crear una imagen PNG.
ean.save(nombre_archivo_salida)	Guarda el código de barras en un archivo PNG con el nombre del alumno

- Preparación del Archivo CSV
  Crea un archivo llamado, por ejemplo, alumnos.csv con el formato solicitado:

  "Alumno 01","1"
  "Alumno 02","2"
  "María García","100"

"Juan Pérez","123456789012"
"Alumna 05","5"

Cada línea representa un alumno, con su nombre entre comillas seguido de su ID también en tre comillas, separados por una coma.

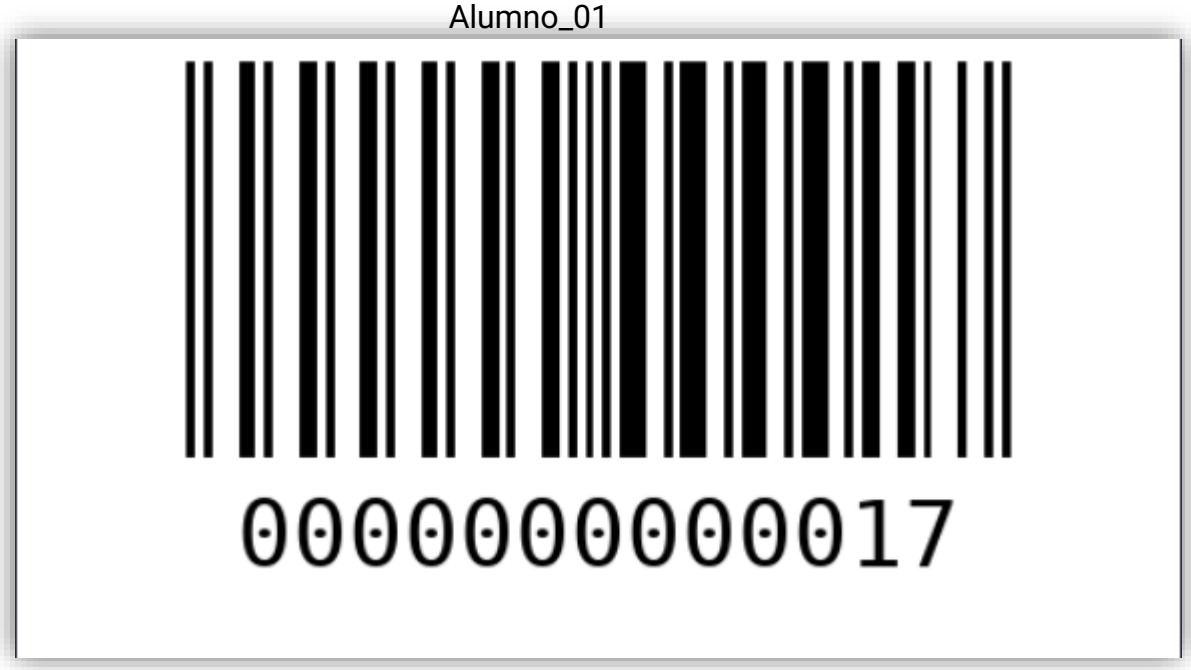
EAN-13: El estándar EAN-13 requiere un número de 12 dígitos (más un dígito de control que se añade automáticamente). El código anterior usa la función formatear_id_para_ean13 para rellenar automáticamente los IDs cortos (como "1" o "100") con ceros a la izquierda hasta completar los 12 dígitos, lo cual es necesario para la generación. Por ejemplo, "1" se convierte en "000000000001".

- Ejecución desde la Consola
  Ejecutar el programa pasando la ruta de tu archivo CSV como argumento:

  python generar_codigos.py alumnos.csv

```
s\UD00 - Actividad Evaluable 02\pregunta9> python generar_codigos.py alumnos.csv
Procesando el archivo CSV: alumnos.csv
✅ Generado: 'Alumno_01.png.png' para el alumno 'Alumno 01' con ID: 1
✅ Generado: 'Alumno_02.png.png' para el alumno 'Alumno 02' con ID: 2
✅ Generado: 'María_García.png.png' para el alumno 'María García' con ID: 100
✅ Generado: 'Juan_Pérez.png.png' para el alumno 'Juan Pérez' con ID: 123456789012
✅ Generado: 'Alumna_05.png.png' para el alumno 'Alumna 05' con ID: 5
```

Alumno_01



PREGUNTA ENTREVISTA 10

Complementando la pregunta anterior y utilizando como base el siguiente ejemplo de código **https://www.geeksforgeeks.org/how-to-make-a-barcode-reader-in-python/** haz un programa que lea todos los ".png" de un directorio y nos diga el nombre de alumno y su ID. (Recuerda, el nombre del alumno es el nombre del ".png" y el ID debes extraerlo del código de barras).

Consejo 01: para usar esta biblioteca, además de instalarla con "**pip3 install pyzbar**", debes instalar la utilidad "**zbar**" que se puede instalar con "**apt install zbar-tools**"

Consejo 02: para utilizar la biblioteca "cv2" (las siglas CV son de Computer Vision), has de instalar **"pip3 install opencv-python"** y **"pip3 install opencv-contrib-python"**

- Instalar las bibliotecas:

```
pip install opencv-python
pip install pyzbar
```

Lectura de códigos de barras con zbarimg

Leer códigos de barras EAN-13 desde imágenes .png generadas previamente, extrayendo el **ID del alumno** codificado en el código de barras, y asociándolo con el **nombre del alumno** (obtenido del nombre del archivo).

Problema inicial con pyzbar

- Se intentó usar la librería pyzbar para decodificar los códigos de barras.
- El entorno Windows requería DLLs externas (libzbar-64.dll, libiconv.dll) que no estaban correctamente enlazadas.
- A pesar de copiar los DLLs en varias ubicaciones, el error persistía:

  FileNotFoundError: Could not find module 'libiconv.dll'

Solución: uso de zbarimg

Se optó por usar zbarimg, una herramienta de línea de comandos incluida en la instalación de ZBar, que permite leer códigos de barras directamente desde imágenes.

Ventajas:

- No requiere integración con DLLs en Python.
- Funciona de forma independiente como ejecutable.
- Compatible con Windows y fácil de invocar desde Python usando subprocess.

Instalación de ZBar y configuración

1. **Descarga**: Se descargó zbar-0.10-setup.exe desde SourceForge.
2. **Instalación**: Se instaló ZBar en el sistema.
3. **Verificación**: Se confirmó que zbarimg estaba disponible ejecutando:
   zbarimg --version
   Resultado: 0.10
4. **Configuración del entorno**: Se agregó la ruta de instalación (C:\Program Files (x86)\ZBar\bin) a la variable de entorno PATH.

```python
# lector_codigo_barra.py

import os
import subprocess

def leer_codigo_barra(imagen_path):
    '''
    Lee el código de barras EAN-13 de una imagen usando zbarimg.
    Args:
        imagen_path (str): Ruta a la imagen del código de barras.
    Returns:
        str: El código de barras leído o None si no se pudo leer.
    '''

    try:
        resultado = subprocess.run(['zbarimg', imagen_path], capture_output=True, text=True)
        salida = resultado.stdout.strip()
        if salida.startswith('EAN-13:'):
```

```
            return salida.split(':')[1]
        except Exception as e:
            print(f"Error al leer {imagen_path}: {e}")
        return None

def procesar_directorio(directorio):
    '''
        Procesa todas las imágenes .png en un directorio para leer códigos de barras.
        Args:
            directorio (str): Ruta al directorio con las imágenes.
    Returns:
        None
    '''
    for archivo in os.listdir(directorio):
        if archivo.lower().endswith('.png'):
            ruta_imagen = os.path.join(directorio, archivo)
            nombre_alumno = os.path.splitext(archivo)[0].replace('_', ' ')
            id_alumno = leer_codigo_barra(ruta_imagen)
            if id_alumno:
                print(f"Alumno: {nombre_alumno} | ID: {id_alumno}")
            else:
                print(f"Alumno: {nombre_alumno} | Código no detectado")

if __name__ == "__main__":
    directorio = input("Introduce la ruta del directorio con los .png: ")
    procesar_directorio(directorio)
```

- Ejecutar en consola : python lector_codigo_barra.py

```
2\pregunta9> python lector_codigo_barra.py
Introduce la ruta del directorio con los .png: C:\Users\
les individuals\UD00 - Actividad Evaluable 02\pregunta9
Alumno: Alumna 05.png | ID: 0000000000055
Alumno: Alumno 01.png | ID: 0000000000017
Alumno: Alumno 02.png | ID: 0000000000024
Alumno: Juan Pérez.png | ID: 1234567890128
Alumno: María García.png | ID: 0000000001007
```

PREGUNTA ENTREVISTA 11

Crea un programa que tenga una "interfaz gráfica" sencilla utilizando "**Kivy**", un entorno multi-plataforma para Python **https://kivy.org**.
Este programa, al arrancar, cargará un archivo ("users.csv") que por cada línea tendrá un usuario y el SHA1 del password, con el siguiente formato:
"user":"passwordinSHA1"
La interfaz gráfica que tendrá sólo dos campos de texto, "usuario" y "contraseña" (en un texto plano, pero oculta al teclear) y un botón "comprobar". Si el usuario existe y la contraseña es correcta, mostrará un mensaje de "OK". De lo contrario, un mensaje "ERROR".

Consejo 01: para leer archivos ".csv" puedes utilizar la biblioteca "csv" o la biblioteca "pandas". Aquí tienes ejemplos:

https://code.tutsplus.com/es/tutorials/how-to-read-and-write-csv-files-in-python--cms-29907

https://pythonbros.com/leer-csv-con-pandas/

Consejo 02: para comprobar si una contraseña en SHA1 es correcta, aprovechando las propiedades de las funciones "hash" **https://es.wikipedia.org/wiki/Funci%C3%B3n_hash** lo que debes hacer es obtener el "hash" de la contraseña en texto plano y comparar ese "hash" obtenido con el "hash" almacenar. Si son iguales, la contraseña es correcta.

Un ejemplo de SHA1 en Python 3 **https://programmerclick.com/article/7363885029/**

Consejo 03: tienes un ejemplo de una interfaz de inicio de sesión en Kivy en

https://gist.github.com/Cheaterman/812203a74f8c552a4918

También este tutorial

https://kivycoder.com/creating-a-login-screen-with-kivymd-python-kivy-gui-tutorial-44/

el vídeo **https://www.youtube.com/watch?v=G-Rp41BzGxg**

PREGUNTA ENTREVISTA 12

Replica este ejemplo de Python 3 para reconocer texto en imágenes. Añade comentarios para explicar su uso.

https://towardsdatascience.com/building-a-simple-text-recognizer-in-python-93e453ddb759

Puede probar con alguna imagen sencilla (sólo texto y un fondo homogéneo) como:

https://www.facebook.com/MumukiOrg/photos/pcb.4322522981176358/4322522757843047/

Consejo 01: para esta prueba, además de las bibliotecas Pillow y PyTesseract instaladas **con "pip3 install Pillow pytesseract",** debes instalar la utilidad "Tesseract" que se puede instalar con **"apt install tesseract-ocr libtesseract-dev"**

- Instalar las bibliotecas necesarias:
  pip install pytesseract

- Instalar Tesseract OCR en Windows
  - Descarga el instalador desde GitHub – Tesseract OCR o directamente desde UB Mannheim.

  - Instala el programa y anota la ruta de instalación, por ejemplo:
    C:\Program Files\Tesseract-OCR\tesseract.exe

- Preparación de la imagen
  Guardar una imagen con texto claro y fondo homogéneo en la misma carpeta del script.
  Se uso como nombre ejemplo.png.

- Corrección del error de idioma
  El error indicaba que faltaba el archivo spa.traineddata.
  Se descargo el archivo desde desde https://github.com/tesseract-ocr/tessdata
  Se lo coloco en la carpeta tessdata dentro de la instalación de Tesseract.

- Ejecución del script
  desde la terminal: python ocr_texto_simple.py

```
s\UD00 - Actividad Evaluable 02\pregunta12> python ocr_texto_simple.py
Texto detectado:
La prueba y el error
forman parte de la
práctica misma de
programar. Por eso, al
aprender programación
también debemos
aprender a lidiar con los
errores para evitar
frustrarnos. No podemos
programar sin aceptar y
aprender del error.
```

PREGUNTA ENTREVISTA 13

Utilizando la biblioteca de Python 3 “requests” y basándose en ejemplos como

https://pythonizando.com/consumir-una-api-rest-con-python-3-usando-requests/

Realiza un programa que consume la API de “Rick and Morty” **https://rickandmortyapi.com/**

Nuestro programa nos permitirá, pidiendo por teclado una especie (atributo “specie”), obtener un listado de qué personajes son de esa especie.

Consejo 01: para instalar "requests" puedes hacerlo con "**pip3 install requests**"

https://4geeks.com/es/how-to/como-consumir-una-api-en-python

- El programa pide al usuario una especie.
- Hace peticiones paginadas a la API de Rick and Morty.
- Filtra los personajes cuya especie coincide con la introducida.
- Muestra el listado por consola.

```
import sys
import requests

def obtener_personajes_por_especie(especie):
    '''
    Obtiene los nombres de los personajes de Rick y Morty
    que pertenecen a una especie específica.

    Args:
        especie (str): La especie de los personajes a buscar.

    Returns:
        list: Lista de nombres de personajes.
```

```
    Raises:
        requests.exceptions.RequestException: Si ocurre un error en la solicitud HTTP.
    '''
    url = f"https://rickandmortyapi.com/api/character/?species={especie}"
    personajes = []

    try:
        respuesta = requests.get(url)
        respuesta.raise_for_status()
        data = respuesta.json()

        if 'results' in data:
            for personaje in data['results']:
                personajes.append(personaje['name'])

    except requests.exceptions.RequestException as e:
        print(f"Error al realizar la solicitud: {e}")

    return personajes

def main():
    """
    Función principal que solicita la especie al usuario e inicia la búsqueda.
    """
    print()
    print("-------------------------------------------------------------")
    print("  Bienvenido al Buscador de Personajes de Rick and Morty por Especie  ")
    print("-------------------------------------------------------------")

    especie_busqueda = input("Ingresa la especie de personaje que deseas buscar (ej: Human, Alien,
Mythological Creature): ").strip()

    if not especie_busqueda:
        print("Debes ingresar una especie para realizar la búsqueda. Saliendo del programa.")
        sys.exit(1)

    personajes = obtener_personajes_por_especie(especie_busqueda)
```

```
    if personajes:
        print(f"\nPersonajes de la especie '{especie_busqueda}':")
        for nombre in personajes:
            print(f"- {nombre}")
    else:
        print(f"No se encontraron personajes de la especie '{especie_busqueda}'.")

    print("\n------------------------------------------------------------")
    print("Búsqueda finalizada.")

if __name__ == "__main__":
    main()


python buscador_rick_morty.py

------------------------------------------------------------
  Bienvenido al Buscador de Personajes de Rick and Morty por Especie
------------------------------------------------------------
Ingresa la especie de personaje que deseas buscar (ej: Human, Alien, Mythological Creature): Human

Personajes de la especie 'Human':
- Rick Sanchez
- Morty Smith
- Summer Smith
- Beth Smith
- Jerry Smith
- Abradolf Lincler
- Adjudicator Rick
- Agency Director
- Alan Rails
- Albert Einstein
- Alexander
- Annie
- Antenna Morty
- Antenna Rick
- Ants in my Eyes Johnson
- Aqua Morty
```

```
- Aqua Rick
- Artist Morty
- Attila Starwar
- Baby Legs


----------------------------------------------------------------
Búsqueda finalizada.
```

PREGUNTA ENTREVISTA 14

El problema de las 8 Reinas es ampliamente conocido

https://es.wikipedia.org/wiki/problema_de_las_ocho_reinas

Vamos a darle una vuelta y vamos a proponer el problema de las N reinas.

Queremos que hagas un programa, que para un número N (4 <= N <= 10) calcule de cuántas formas **diferentes** se pueden colocar N reinas en un tablero de NxN dimensiones.

NOTA IMPORTANTE: no se admiten precálculos, el programa debe calcular las soluciones.

Para probar, los resultados deberían ser:

Tamaño N	Soluciones diferentes	Soluciones totales (sin eliminar repeticiones)
4	1	2
5	2	10
6	1	4
7	6	40
8	12	92
9	46	352
10	92	724
11	341	2,680
12	1,787	14,200

Para simplificar el problema, su programa sólo tendrá que calcular las "Soluciones totales" del problema de las N reinas (sin eliminar repeticiones)

PREGUNTA ENTREVISTA 15

Haz un programa que lea de un archivo que se pase como parámetro, un listado de palabras (una palabra por línea). Estas palabras será un listado de "palabras prohibidas".
Utilizando la biblioteca https://pyperclip.readthedocs.io/en/latest/, realice un programa que cuando copias algo del portapapeles, analice el texto copiado y sustituya de las palabras prohibidas (sin distinguir mayúsculas y minúsculas) los caracteres de las palabras prohibidas en el listado por * y el vuelva a copiar en el portapapeles.

Ejemplo de funcionamiento:

Llamamos al programa "**programa.py lista.txt**"
El archivo "lista.txt" contento las palabras:
Hola
pYThon

Con el programa en marcha, si copie en el portapapeles la frase "HOLA Sergi viva PYTHON",

debería modificar al portapapeles esta frase y dejarla como "**** Sergi viva ******", de forma que cuando pegamos, se pegue esta segunda versión.

Consejo 01: Puede instalarla con **"pip3 install pyperclip".**

Si le da "Not implemented Error", instale los siguientes programas:

"sudo apt-get install xsel"

"sudo apt-get install xclip"

```python
#programa.py

import sys
import pyperclip
import time
import re

def cargar_palabras_prohibidas(nombre_archivo):
    """
    Carga las palabras prohibidas desde un archivo.
    Todas las palabras se convierten a minúsculas para una comparación
    insensible a mayúsculas y minúsculas.

    Args:
        nombre_archivo (str): Ruta al archivo que contiene las palabras prohibidas.

    Returns:
        set: Un conjunto de palabras prohibidas en minúsculas.
    """
    try:
        with open(nombre_archivo, 'r', encoding='utf-8') as archivo:
            # strip() elimina espacios en blanco y saltos de línea
            # lower() convierte a minúsculas
            palabras = {linea.strip().lower() for linea in archivo if linea.strip()}
        print(f"Palabras prohibidas cargadas: {len(palabras)} ({', '.join(palabras)})")
        return palabras
    except FileNotFoundError:
        print(f"Error: El archivo '{nombre_archivo}' no fue encontrado.")
        sys.exit(1)
    except Exception as e:
        print(f"Ocurrió un error al leer el archivo: {e}")
        sys.exit(1)

def filtrar_texto(texto, palabras_prohibidas):
    """
```

```
    Analiza el texto y reemplaza cualquier palabra prohibida con asteriscos (*).
    Mantiene la capitalización y formato del texto original.

    Args:
        texto (str): El texto a analizar.
        palabras_prohibidas (set): Conjunto de palabras prohibidas en minúsculas.
    Returns:
        str: El texto modificado con palabras prohibidas reemplazadas.
        bool: Indica si el texto fue modificado.
    """
    # Expresión regular para encontrar cualquier secuencia que NO sea letra/número/guion bajo
    # Usamos re.split para dividir el texto y mantener los delimitadores
    # re.IGNORECASE asegura que la comparación sea insensible a mayúsculas/minúsculas

    # Este patrón encuentra las 'palabras' (secuencias de letras y números)
    # y los 'separadores' (secuencias de otros caracteres como espacios, puntuación, etc.)
    partes = re.split(r'(\b\w+\b)', texto, flags=re.IGNORECASE)

    texto_filtrado = []
    modificado = False

    for parte in partes:
        # Si la parte es una palabra (no un separador vacío)
        if parte.strip() and parte.isalnum():
            # Comparamos la palabra en minúsculas con nuestro conjunto de prohibidas
            if parte.lower() in palabras_prohibidas:
                # Reemplazamos por asteriscos de la misma longitud
                texto_filtrado.append('*' * len(parte))
                modificado = True
                print(f"  Reemplazada: '{parte}'")
            else:
                texto_filtrado.append(parte)
        else:
            # Los separadores (espacios, puntuación) se añaden sin cambios
            texto_filtrado.append(parte)

    return "".join(texto_filtrado), modificado
```

```
def main():
    if len(sys.argv) < 2:
        print("Uso: python programa.py <archivo_de_palabras_prohibidas>")
        sys.exit(1)

    nombre_archivo = sys.argv[1]

    palabras_prohibidas = cargar_palabras_prohibidas(nombre_archivo)

    ultimo_texto = pyperclip.paste()
    print("Portapapeles inicial: '", ultimo_texto[:30], "...'")


    print("\nEl programa está monitoreando el portapapeles (presiona Ctrl+C para detener).")

    while True:
        try:
            texto_actual = pyperclip.paste()

            if texto_actual != ultimo_texto:
                print(f"\nNuevo texto detectado: '{texto_actual}'")

                # Filtrar el texto
                texto_filtrado, fue_modificado = filtrar_texto(texto_actual, palabras_prohibidas)

                if fue_modificado:
                    # Copiar el texto modificado al portapapeles
                    pyperclip.copy(texto_filtrado)
                    print(f"Texto modificado y copiado al portapapeles: '{texto_filtrado}'")

                    ultimo_texto = texto_filtrado
                else:
                    print(" No se encontraron palabras prohibidas. Texto sin cambios.")
                    ultimo_texto = texto_actual

            time.sleep(0.1)

        except Exception as e:
```

```
        print(f"\nOcurrió un error inesperado: {e}")
        time.sleep(1)

if __name__ == "__main__":
    main()
```

lista.txt

Hola

pYThon

malapalabra

Ejecutar el programa programa.py desde tu terminal, pasando el nombre del archivo de la lista como argumento:

python programa.py lista.txt

Mensaje indicando que el programa está monitoreando el portapapeles.

```
2> python programa.py lista.txt
Palabras prohibidas cargadas: 2 (hola, python)
Portapapeles inicial: ' Ejecuta el programa programa.p ...'

El programa está monitoreando el portapapeles (presiona Ctrl+C para detener).
```

Copia la siguiente frase desde cualquier lugar (navegador, documento, etc.) a tu portapapeles:

HOLA Sergi viva PYTHON, esto es una malaPalabra.

```
Nuevo texto detectado: 'HOLA Sergi viva PYTHON, esto es una malaPalabra.'
  Reemplazada: 'HOLA'
  Reemplazada: 'PYTHON'
  Reemplazada: 'malaPalabra'
Texto modificado y copiado al portapapeles: '**** Sergi viva ******, esto es una **********.'
```

BIBLIOGRAFIA

- Apuntes del Profesor.
- Python para todos. Explorando la información con Python 3. Charles R. Severance
- https://docs.python.org/es/3/library/copy.html
- https://www.delftstack.com/
- https://apuntes.de/python/creacion-de-modulos-y-paquetes-en-python-modularidad-y-organizacion/#gsc.tab=0
- https://www.datacamp.com/es/tutorial/sqlalchemy-tutorial-examples
- https://j2logo.com/python/python-requests-peticiones-http/
- https://www.youtube.com/watch?v=Kbc-Qr318gs